```
/* ================================================================
 * 文件：server/app.js
 * ================================================================ */
/**
 * app.js —— 应用入口
 * 一个进程同时提供：REST API（/api/**） + 前端静态资源（/）
 */
'use strict';

const express = require('express');
const path = require('path');
const config = require('./config');
const { ok, fail } = require('./middleware/auth');
const { getChain } = require('./services/anchor');

const app = express();

/* ---------------- 进程级兜底 ----------------
 * Node 15+ 默认会把未处理的 Promise 拒绝升级为进程退出。
 * 演示现场（以及无人值守的长跑）绝不能让一个漏 catch 的异步分支把节点带走，
 * 因此这里统一记日志并让进程继续服务。
 */
process.on('unhandledRejection', (reason) => {
  console.error('[UNHANDLED_REJECTION]', reason && reason.stack ? reason.stack : reason);
});
process.on('uncaughtException', (err) => {
  console.error('[UNCAUGHT_EXCEPTION]', err && err.stack ? err.stack : err);
});

/* ---------------- 基础中间件 ---------------- */
app.use(express.json({ limit: '2mb' }));
app.use(express.urlencoded({ extended: true }));

/* 跨域策略（安全默认）
 *   · 配置了 CORS_ORIGINS      → 白名单模式，只放行名单内来源；
 *   · 未配置 且为本地/内网开发 → 回显请求来源，便于前后端分端联调；
 *   · 未配置 且处于生产/云平台 → 不下发任何跨域放行头。
 * 为什么生产环境要"什么都不发"：本平台前后端同端口部署，同源请求本来就不需要
 * 任何 CORS 响应头，功能不受影响；而一旦回显任意 Origin，配合
 * Access-Control-Allow-Credentials: true，等于允许任意第三方站点携带用户凭据
 * 读取接口数据 —— 这正是线上旧版本暴露的问题。 */
const CORS_ORIGINS = String(process.env.CORS_ORIGINS || '')
  .split(',').map((s) => s.trim()).filter(Boolean);
app.use((req, res, next) => {
  const origin = req.headers.origin;
  let allowOrigin = null;
  if (CORS_ORIGINS.length) {
    if (origin && CORS_ORIGINS.includes(origin)) allowOrigin = origin;
  } else if (!config.isProd) {
    allowOrigin = origin || '*';
```

```
  }
  if (allowOrigin) {
    res.setHeader('Access-Control-Allow-Origin', allowOrigin);
    res.setHeader('Access-Control-Allow-Credentials', 'true');
    res.setHeader('Vary', 'Origin');
  }
  res.setHeader('Access-Control-Allow-Methods', 'GET,POST,PUT,DELETE,OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type,Authorization');
  if (req.method === 'OPTIONS') return res.sendStatus(204);
  next();
});

// 访问日志
app.use((req, res, next) => {
  const t = Date.now();
  res.on('finish', () => {
    if (req.path.startsWith('/api')) {
      console.log(`${new Date().toLocaleTimeString('zh-CN')} ${req.method} ${req.path} ${res.statusCode} ${Date.now() - t}ms
    }
  });
  next();
});

/* ---------------- 生产加固中间件 ----------------
 * 结构化日志 + 监控指标 + 灰度路由 —— 三件套构成"生产加固"基础设施：
 *   · structured-log：JSON 日志 + 环形缓冲（/api/ops/logs 可查）
 *   · monitor：Prometheus 指标 + 阈值告警（/api/ops/metrics, /api/ops/alerts）
 *   · canary：灰度发布功能开关（?canary=1 或 X-Canary header）
 */
app.use(require('./middleware/structured-log').requestLogger);
app.use(require('./middleware/monitor').middleware);
app.use(require('./middleware/canary').middleware);

/* ---------------- 业务路由 ---------------- */
app.use('/api/auth', require('./routes/auth'));
app.use('/api/enterprise', require('./routes/enterprise'));
app.use('/api/verifier', require('./routes/verifier'));
app.use('/api/regulator', require('./routes/regulator'));
app.use('/api/chain', require('./routes/chain'));
app.use('/api/consortium', require('./routes/consortium'));
app.use('/api/dynamic', require('./routes/dynamic'));
app.use('/api/credit', require('./routes/credit'));
app.use('/api/public', require('./routes/public'));
app.use('/api/ops', require('./routes/ops'));

app.get('/api/health', async (req, res) => {
  try {
    const chain = await getChain();
    const stats = await chain.stats();
    ok(res, {
```

```
      status: 'UP', time: new Date().toISOString(),
      chainHeight: stats.height, node: stats.node, difficulty: stats.difficulty,
      consensus: stats.consensusMode || config.chain.consensusMode,
      runMode: config.isCloud ? 'cloud' : (config.isProd ? 'production' : 'development'),
      demoAccounts: config.demoAccounts,
    });
  } catch (e) {
    fail(res, e.message, 500);
  }
});

/* ---------------- 静态资源 ---------------- */
app.use(express.static(config.paths.public, { extensions: ['html'] }));

// SPA 兜底
app.get(/^\/(?!api\/).*/, (req, res) => {
  res.sendFile(path.join(config.paths.public, 'index.html'));
});

/* ---------------- 统一异常处理 ---------------- */
app.use((req, res) => fail(res, '接口不存在', 404));
app.use((err, req, res, next) => {
  console.error('[ERROR]', req.method, req.originalUrl, err.message);
  if (res.headersSent) return next(err);
  fail(res, err.message || '服务器内部错误', err.status || 500);
});

/* ---------------- 启动 ---------------- */

/** 启动前安全自检：关键密钥若为内置默认值，生产环境直接拒绝启动 */
function assertSecureConfig() {
  const problems = [];
  if (!process.env.JWT_SECRET) problems.push('JWT_SECRET 未设置（正在使用内置默认密钥，令牌可被伪造）');
  if (config.isProd && !process.env.DB_PASSWORD) problems.push('DB_PASSWORD 未设置（数据库使用默认口令）');
  if (config.isProd && !process.env.KEYSTORE_SECRET) problems.push('KEYSTORE_SECRET 未设置（链上私钥库将明文落盘）');
  if (!problems.length) return;
  const msg = '⚠ 安全配置检查未通过：\n  - ' + problems.join('\n  - ');
  if (config.isProd) {
    console.error(msg + '\n生产环境禁止以不安全配置启动，进程已中止。');
    process.exit(1);
  }
  console.warn(msg + '\n（当前为非生产环境，仅告警；上线前请务必配置上述环境变量）');
}

async function bootstrap() {
  assertSecureConfig();

  /* ── 启动顺序很关键：先把 HTTP 端口监听起来 ──
     数据库 / 联盟链 / 动态核算的初始化全部放进 warmup()，在 listen 之后异步执行。
     原因：云平台（Render 等）的健康检查依赖端口尽早可用；若在这里同步 await 远程
```

```
    数据库（跨区连接可能耗时数分钟），新实例会被判定「启动过慢」，结果是部署虽然
    显示成功、流量却没有切到新实例 —— 表现为线上仍是上一版代码。 */
  const server = app.listen(config.port, config.host, () => {
    console.log('─'.repeat(64));
    console.log('  碳链通 CarbonChain Hub 已启动');
    console.log(`  访问地址: http://localhost:${config.port}`);
    console.log(`  API 前缀: http://localhost:${config.port}/api`);
    console.log(`  运行模式: ${config.isCloud ? '云平台' : (config.isProd ? '生产' : '开发')}` +
      ` · 演示账号接口${config.demoAccounts ? '开放' : '关闭'}` +
      ` · 跨域${CORS_ORIGINS.length ? `白名单(${CORS_ORIGINS.length})` : (config.isProd ? '仅同源' : '宽松')}`);
    console.log('─'.repeat(64));
  });

  /* 挂载工业物联网 WebSocket 实时通道（容错：ws 库未装则跳过，前端降级归档模式）
     渠道：/ws/telemetry —— 推送 CEMS 烟气实时流 / 车间电表 / 联盟链出块事件 */
  try {
    const telemetry = require('./routes/telemetry');
    telemetry.attach(server);
  } catch (e) { console.warn('⚠ WebSocket 通道挂载跳过：', e.message); }

  /* 后台预热：幂等、失败仅告警，绝不阻塞端口可用性 */
  warmup().catch((e) => {
    console.error('⚠ 后台初始化异常（服务继续运行）：', e && e.message ? e.message : e);
  });
}

/** 链 / 联盟链 / 动态核算的幂等预热 —— 在 HTTP 已监听之后异步执行 */
async function warmup() {
  try {
    const chain = await getChain();
    const stats = await chain.stats();
    console.log('⛓  区块链节点已就绪：高度=%d 区块=%d 交易=%d 共识=%s',
      stats.height, stats.totalBlocks, stats.confirmed, stats.consensusMode || 'POA');
    // 联盟链：幂等注册初始成员节点，并把各节点链副本同步到最新高度
    try {
      const cons = require('./services/consortium');
      await cons.ensureSeed();
      await cons.syncAll();
      const rs = await cons.replicaStatus();
      console.log('🔗 联盟链已就绪：授权节点 %d 个，副本一致节点 %d/%d',
        rs.nodes.filter((n) => n.status === 'ACTIVE').length, rs.consistentNodes, rs.totalNodes);
    } catch (e) {
      console.error('⚠ 联盟节点初始化跳过：', e.message);
    }
    // 动态核算：自举实测数据 + 启动准实时采数调度
    try {
      const dyn = require('./services/dynamic');
      const ov = await dyn.bootstrap();
      dyn.startScheduler();
      console.log('📈 动态核算引擎已启动：实测读数 %d 条，覆盖企业 %d 家，高/中风险 %d/%d',
```

```
        ov.readings.total, ov.summary.enterprises, ov.summary.highRisk, ov.summary.mediumRisk);
    } catch (e) {
      console.error('⚠ 动态核算初始化跳过：', e.message);
    }
  } catch (e) {
    console.error('⚠ 区块链初始化失败（数据库未初始化？请先执行 npm run initdb）：', e.message);
  }

}

if (require.main === module) bootstrap();

module.exports = app;
/* 供自定义启动器（server/scripts/start-prod.js）在完成数据库自检后调用 */
module.exports.bootstrap = bootstrap;

/* ================================================================
 * 文件：server/config.js
 * ================================================================ */
/**
 * config.js —— 全局配置
 * 全部支持环境变量覆盖，便于本地运行与云端部署共用一套代码。
 */
'use strict';

const path = require('path');

/**
 * 是否启用 TLS 连接数据库。
 * DB_SSL=true            开启（校验证书，适用于正规云数据库）
 * DB_SSL=require|true    同上
 * DB_SSL=no-verify       开启但不校验证书（部分免费实例不提供 CA，TiDB Cloud Serverless 常见）
 * 未设置 / false         关闭（本地 MySQL 默认）
 */
function dbSsl() {
  const v = String(process.env.DB_SSL || '').toLowerCase();
  if (['no-verify', 'noverify', 'insecure'].includes(v)) return { rejectUnauthorized: false };
  if (['true', '1', 'yes', 'require', 'required'].includes(v)) return { rejectUnauthorized: true };
  return undefined;
}

/**
 * 解析数字型环境变量，带"空字符串/非法值"兜底。
 * 注意：Number('') === 0，直接写 Number(process.env.PORT || 8300) 在
 * 某些平台把变量注入为空串时会导致监听端口 0（随机端口），从而部署失败。
 */
function num(v, fallback) {
  const n = Number(v);
  return v === undefined || v === null || v === '' || !Number.isFinite(n) ? fallback : n;
}
```

```
/* 托管云平台探测：Render / Heroku / Fly / Vercel / Railway / Cloud Run 等
 * 会自动注入下列环境变量之一。用它判定"运行在线上"，比依赖使用者记得设置
 * NODE_ENV=production 更可靠 —— 安全默认值（关闭演示账号接口、收紧跨域）
 * 据此生效，避免出现"忘了设环境变量 = 线上裸奔"。 */
const IS_CLOUD = !!(process.env.RENDER || process.env.RENDER_EXTERNAL_URL
  || process.env.RENDER_SERVICE_ID || process.env.DYNO
  || process.env.FLY_APP_NAME || process.env.VERCEL
  || process.env.RAILWAY_ENVIRONMENT || process.env.K_SERVICE);

/** 生产语义：显式声明 production，或运行在托管云平台上 */
const IS_PROD = process.env.NODE_ENV === 'production' || IS_CLOUD;

module.exports = {
  /* ---- 运行环境 ---- */
  isCloud: IS_CLOUD,
  isProd: IS_PROD,
  /* 演示账号接口开关（安全默认：线上关闭）。
   * 显式 DEMO_ACCOUNTS=true / false 优先；未设置时，只有非生产环境开放。
   * 该接口会明文返回演示口令，因此绝不能在公网环境默认开放。 */
  demoAccounts: process.env.DEMO_ACCOUNTS === 'true'
    || (process.env.DEMO_ACCOUNTS === undefined && !IS_PROD),

  /* ---- HTTP 服务 ---- */
  port: num(process.env.PORT, 8300),
  /* 云平台（Render 等）要求监听 0.0.0.0 才能接收外部请求；本地也适用。 */
  host: process.env.HOST || '0.0.0.0',

  /* ---- MySQL ---- */
  db: {
    host: process.env.DB_HOST || '127.0.0.1',
    port: num(process.env.DB_PORT, 3306),
    user: process.env.DB_USER || 'root',
    password: process.env.DB_PASSWORD || '123456',
    database: process.env.DB_NAME || 'carbon_chain',
    charset: 'utf8mb4',
    waitForConnections: true,
    connectionLimit: 10,
    /* 时区固定为 UTC（'Z'），不要用 'local'。
     * 原因：区块哈希的输入包含 timestamp（绝对秒数），任何一次
     * "DATETIME 字面量 ↔ 秒数" 的换算都必须在所有机器上得到同一个答案。
     *   · 驱动侧：timezone='Z' 让驱动不做本地时区换算；
     *   · 服务端侧：chain.js 在每次读写前 `SET time_zone = '+00:00'`，
     *     让 MySQL 与 TiDB 的 UNIX_TIMESTAMP() 都按 UTC 解释 DATETIME。
     * 两者配对后，「本机 MySQL(CST)」与「TiDB Cloud(UTC)」反解出的秒数完全一致，
     * 同一份数据在两端都能通过全链校验。缺任何一半都会导致整条链校验失败。 */
    timezone: 'Z',
    dateStrings: ['DATE', 'DATETIME'],
    /* 云端托管 MySQL（TiDB Cloud / PlanetScale / Aiven 等）强制要求 TLS 加密连接。
     * 本地 MySQL 默认关闭，因此这里用环境变量开关，保持一套代码两处通用。 */
```

```
    ssl: dbSsl(),
  },

  /* ---- 认证 ---- */
  jwt: {
    secret: process.env.JWT_SECRET || 'carbonchain-hub-secret-2026-change-me',
    expiresIn: 8 * 3600, // 秒
  },

  /* ---- 区块链 ---- */
  chain: {
    consensusMode: String(process.env.CHAIN_CONSENSUS || 'POA').toUpperCase(), // POA 联盟链授权出块 / POW 历史兼容
    difficulty: Number(process.env.CHAIN_DIFFICULTY || 4),
    autoMine: process.env.CHAIN_AUTOMINE !== 'false',
    batchSize: Number(process.env.CHAIN_BATCH || 1),
    nodeName: process.env.CHAIN_NODE || 'node-carbon-01',
  },

  /* ---- 路径 ---- */
  paths: {
    root: path.resolve(__dirname, '..'),
    public: path.resolve(__dirname, '..', 'public'),
    schema: path.resolve(__dirname, '..', 'db', '01_schema.sql'),
    keystore: path.resolve(__dirname, 'keystore.json'),
  },

  /* ---- 业务参数 ---- */
  biz: {
    quotaPrice: 68.5,       // 碳配额基准价(元/吨)
    defaultQuotaYear: 2026,
  },
};

/* ================================================================
 * 文件：server/db.js
 * ================================================================ */
/**
 * db.js —— MySQL 连接池 + 便捷查询封装
 */
'use strict';

const mysql = require('mysql2/promise');
const config = require('./config');

const pool = mysql.createPool({
  ...config.db,
  multipleStatements: false,
});

/** 普通查询 */
```

```
async function query(sql, params = []) {
  const [rows] = await pool.query(sql, params);
  return rows;
}

/** 取单行 */
async function one(sql, params = []) {
  const rows = await query(sql, params);
  return rows.length ? rows[0] : null;
}

/** 取单值 */
async function value(sql, params = []) {
  const row = await one(sql, params);
  if (!row) return null;
  return Object.values(row)[0];
}

/** 事务执行 */
async function tx(fn) {
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    const result = await fn(conn);
    await conn.commit();
    return result;
  } catch (e) {
    await conn.rollback();
    throw e;
  } finally {
    conn.release();
  }
}

/**
 * 分页查询辅助：自动拼 LIMIT，并返回总数
 * @param {string} baseSql 不含 LIMIT 的 SQL（含 WHERE/ORDER BY）
 * @param {Array} params
 * @param {{page:number,size:number}} page
 */
async function paginate(baseSql, params, page = {}) {
  const p = Math.max(Number(page.page) || 1, 1);
  const s = Math.min(Math.max(Number(page.size) || 10, 1), 200);
  const rows = await query(`${baseSql} LIMIT ${s} OFFSET ${(p - 1) * s}`, params);
  const countSql = `SELECT COUNT(*) AS total FROM (${baseSql}) _t`;
  const total = Number(await value(countSql, params)) || 0;
  return { rows, total, page: p, size: s, pages: Math.ceil(total / s) || 1 };
}

module.exports = { pool, query, one, value, tx, paginate };
```

```
/* ================================================================
 * 文件：server/blockchain/chain.js
 * ================================================================ */
/**
 * chain.js —— 联盟链内核（PoA 授权证明）
 * ----------------------------------------------------------------
 * 本文件实现了一条可直接运行、可被真实业务调用的**联盟链**内核，包含：
 *
 *   · 哈希链结构       block.hash = SHA256(index|timestamp|merkleRoot|prevHash|consensus|proposer|nonce)
 *   · PoA 授权出块     仅"联盟授权节点"具备出块权，按权重轮值出块，无需挖矿
 *   · 出块签名         出块节点对区块头做 ECDSA(secp256k1) 签名，任何节点可验签
 *   · 成员准入         出块权来自 chain_node 白名单（PENDING/ACTIVE/REVOKED）
 *   · Merkle 树        区块内所有交易摘要收敛为一个根
 *   · 交易池           业务动作先入池，打包后落块
 *   · ECDSA 存证签名   每笔交易由发起方私钥签名，节点可验签
 *   · 链完整性校验     逐块重算哈希 + 校验前向引用 + 校验出块签名 + 校验 Merkle 根
 *   · 双共识兼容       历史 PoW 块仍按工作量证明校验（向后兼容，便于迁移）
 *
 * 为什么是"联盟链"而不是公有链：
 *   参与方是**许可制**的成员机构（监管 / 核查 / 企业 / 审计），各自运行一个节点、
 *   各自持有一份全量链副本；出块权由联盟授予，共识采用 PoA（授权证明）而非挖矿。
 *   这样即使某一方的数据库被完全攻陷，其它节点仍能凭各自副本发现并拒绝被篡改的链。
 *
 * @module blockchain/chain
 */
'use strict';

const cu = require('./crypto-utils');
const keystore = require('./keystore');

/** 创世区块固定前向哈希 */
const GENESIS_PREV_HASH = cu.ZERO.repeat(64);

class Blockchain {
  /**
   * @param {object} db  MySQL 连接池
   * @param {object} opts { consensusMode, difficulty, autoMine, batchSize, nodeName }
   *   consensusMode: 'POA'（默认，联盟链授权出块）/ 'POW'（历史兼容）
   */
  constructor(db, opts = {}) {
    this.db = db;
    this.consensusMode = String(opts.consensusMode || process.env.CHAIN_CONSENSUS || 'POA').toUpperCase();
    this.difficulty = opts.difficulty || 4;   // 仅 POW 模式使用
    this.autoMine = opts.autoMine !== false;
    this.batchSize = opts.batchSize || 1;
    this.nodeName = opts.nodeName || 'node-carbon-01';
    this.blocks = [];       // 内存链（已确认）
    this.pool = [];         // 交易池（待打包）
    this.mining = false;    // 出块锁，防止并发出块
```

```
    this.ready = false;
    /* 链数据「变更版本号」：任何一次写入（出块 / 篡改 / 修复 / 重新装载）都自增。
     * 用途见 validateChainCached()：全链校验的缓存键必须包含本字段，
     * 否则「篡改后链高未变」会让缓存把一个已被破坏的链误报为 valid=true。 */
    this._rev = 0;
    this._validateCache = null; // { key, at, value }
  }

  /** 标记链数据已变更（使全链校验缓存失效） */
  _bumpRev() {
    this._rev++;
    this._validateCache = null;
  }

  /* ================================================================
   * 初始化
   * ================================================================ */

  /**
   * 从数据库装载链与交易池；链为空时创建创世块
   *
   * ⚠ 关于时间戳与时区（本文件最关键的一处约定）
   * ----------------------------------------------------------------
   * 区块哈希的输入包含 timestamp（绝对秒数），因此"从 DATETIME 反解出秒数"
   * 这一步必须在**任何机器上都得到同一个答案**。原实现用
   *   UNIX_TIMESTAMP(block_time)
   * 但 MySQL 的 UNIX_TIMESTAMP() 是按【会话时区】解释 DATETIME 的：
   *   · 本机 MySQL 会话时区 = +08:00 → 2026-01-01 10:00 视作 02:00Z
   *   · TiDB Cloud 会话时区  = +00:00 → 同一个字面量视作 10:00Z
   * 结果同一个字面量在两端反解出相差 28800 秒的 timestamp，重算哈希必然失败。
   * 解法两步（缺一不可）：
   *   (1) 连接建立后立刻 `SET time_zone = '+00:00'`，让两端会话统一按 UTC 解释；
   *   (2) 写入时不再做本地→UTC 的换算，直接把秒数按 UTC 格式化成字面量。
   * 这样"写进去的字符串"与"读出来的秒数"在 MySQL 与 TiDB 上完全一致。
   */
  static TIME_ZONE_UTC = true;

  async init() {
    await this.syncTimeZone();
    const [rows] = await this.db.query(
      'SELECT block_index, block_hash, prev_hash, merkle_root, nonce, difficulty, tx_count, miner, size_bytes, mine_time, UN
    );
    if (rows.length === 0) {
      await this.mineBlock({ miner: 'GENESIS', genesis: true });
    } else {
      this.blocks = rows.map((r) => this._rowToBlock(r));
    }

    const [pend] = await this.db.query(
      "SELECT * FROM chain_tx WHERE status = 'PENDING' ORDER BY id ASC"
```

```
    );
    this.pool = pend.map((r) => this._rowToTx(r));
    this.ready = true;
    this._bumpRev(); // 首次装载视为一次变更，丢弃可能存在的陈旧校验缓存
    return this;
  }

  /**
   * 把当前连接的会话时区固定为 UTC。
   * 池化连接下必须逐条连接生效，故先取一条连接执行；mysql2 每次从池里取到的连接
   * 都会带着初始化指令（在 pool 上用 connectionAttributes/initSql 不可用时，这里
   * 采用"每次查询前确保"的轻量策略：连接池是长连接，设一次即可长期生效；
   * 若中途因超时重连，这里都会被 init() / 校验入口再次调用覆盖）。
   */
  async syncTimeZone() {
    try {
      await this.db.query("SET time_zone = '+00:00'");
    } catch (e) {
      // 个别托管库不允许改会话时区（极少见）：此时退化为依赖 dateStrings 与写入格式化，
      // 不影响本机 MySQL，也不阻断启动。
      this._tzWarned = true;
    }
    return this;
  }

  _rowToBlock(r) {
    return {
      index: Number(r.block_index),
      hash: r.block_hash,
      prevHash: r.prev_hash,
      merkleRoot: r.merkle_root,
      nonce: Number(r.nonce),
      difficulty: Number(r.difficulty),
      consensus: r.consensus || 'POW',
      proposer: r.proposer || null,
      proposerSig: r.proposer_sig || null,
      txCount: Number(r.tx_count),
      miner: r.miner,
      sizeBytes: Number(r.size_bytes),
      mineTime: Number(r.mine_time),
      // 区块时间戳统一使用「秒」，与 MySQL DATETIME 精度对齐，
      // 否则毫秒部分在落库时被截断，重算哈希必然不一致。
      timestamp: Number(r.ts),
    };
  }

  _rowToTx(r) {
    return {
      txId: r.tx_id,
      txType: r.tx_type,
```

```
    bizNo: r.biz_no,
    blockIndex: r.block_index === null ? null : Number(r.block_index),
    fromAddress: r.from_address,
    toAddress: r.to_address,
    payloadHash: r.payload_hash,
    payload: r.payload_json ? safeParse(r.payload_json) : null,
    signature: r.signature,
    pubKey: r.pub_key,
    nonce: Number(r.nonce),
    fee: Number(r.gas_fee),
    status: r.status,
    createdAt: new Date(r.created_at).getTime(),
  };
}

/* ================================================================
 * 区块哈希与 PoW
 * ================================================================ */

/**
 * 计算区块头哈希（按共识类型分派，保证历史 PoW 块与新区块都可复现）
 *   · PoA: SHA256(index|timestamp|merkleRoot|prevHash|consensus|proposer|nonce)
 *   · PoW: SHA256(index|timestamp|merkleRoot|prevHash|difficulty|nonce)
 */
static calcHash(b) {
  const consensus = String(b.consensus || 'POW').toUpperCase();
  if (consensus === 'POA') {
    return cu.sha256([
      b.index, b.timestamp, b.merkleRoot, b.prevHash, 'POA', b.proposer || '', b.nonce,
    ].join('|'));
  }
  return cu.sha256([
    b.index, b.timestamp, b.merkleRoot, b.prevHash, b.difficulty, b.nonce,
  ].join('|'));
}

/**
 * 出块签名的待签内容（区块头规范化序列化）。
 * 出块节点用自己的私钥对它签名，任何节点都能用其公钥验签，
 * 从而证明"这个块确实是本轮被授权的节点出的"，且区块头未被改动。
 */
static signingPayload(b) {
  return cu.stableStringify({
    index: b.index, timestamp: b.timestamp, merkleRoot: b.merkleRoot,
    prevHash: b.prevHash, consensus: 'POA', proposer: b.proposer || '', nonce: b.nonce,
  });
}

/**
 * 绝对秒数 → UTC 的 'YYYY-MM-DD HH:mm:ss' 字面量。
```

```
   * 与 `SET time_zone = '+00:00'` 配对使用：写入即 UTC，读出按 UTC 解释，
   * 于是 timestamp 的往返在任何服务器时区下都恒等。
   * @param {number} sec epoch 秒
   */
  static utcDateString(sec) {
    const d = new Date(Number(sec) * 1000);
    const p = (n) => String(n).padStart(2, '0');
    return `${d.getUTCFullYear()}-${p(d.getUTCMonth() + 1)}-${p(d.getUTCDate())} `
      + `${p(d.getUTCHours())}:${p(d.getUTCMinutes())}:${p(d.getUTCSeconds())}`;
  }

  /**
   * 工作量证明：暴力搜索 nonce，使区块哈希以 difficulty 个 0 开头
   * @returns {{nonce:number, hash:string, hashRate:number, elapsed:number}}
   */
  static proofOfWork(header) {
    const target = cu.ZERO.repeat(header.difficulty);
    const start = Date.now();
    let nonce = header.nonce || 0;
    let hash = '';
    for (;;) {
      hash = Blockchain.calcHash({ ...header, nonce });
      if (hash.startsWith(target)) break;
      nonce += 1;
    }
    const elapsed = Math.max(Date.now() - start, 1);
    return { nonce, hash, hashRate: Math.round(nonce / (elapsed / 1000)) || nonce, elapsed };
  }

  /** 校验区块自身（哈希、PoW、结构） */
  static isBlockValid(block) {
    const errors = [];
    const expect = Blockchain.calcHash(block);
    if (expect !== block.hash) errors.push(`区块#${block.index} 哈希不匹配（数据已被改动）`);
    const target = cu.ZERO.repeat(block.difficulty);
    if (!String(block.hash).startsWith(target)) errors.push(`区块#${block.index} 不满足 PoW 难度 ${block.difficulty}`);
    if (block.index > 0 && block.prevHash !== (block.prevHash || '')) {
      // 占位，链式校验在 validateChain 中做
    }
    return { ok: errors.length === 0, errors };
  }

  /* ================================================================
   * 交易：构造 → 签名 → 入池 → 打包
   * ================================================================ */

  /**
   * 构造并签名一笔存证交易
   * @param {object} p
   *   p.txType     业务类型
```

```
   *   p.bizNo      业务单号
   *   p.fromAddress 发起方地址
   *   p.toAddress  接收方地址（可选）
   *   p.payload    业务数据对象（会被哈希并快照存证）
   *   p.privateKey 发起方私钥 PEM（用于 ECDSA 签名）
   *   p.publicKey  发起方公钥 PEM
   *   p.fee        存证手续费
   * @returns {object} 交易对象
   */
  buildTx(p) {
    const payloadHash = cu.hashObject(p.payload || {});
    const nonce = Date.now() + Math.floor(Math.random() * 1000);
    const txId = cu.sha256(
      [p.fromAddress || 'SYSTEM', nonce, payloadHash, p.txType, p.bizNo || ''].join('|')
    ).slice(0, 64);

    // 签名内容：交易核心字段（不含签名字段自身），保证任何字段被改都会验签失败
    const signContent = cu.stableStringify({
      txId,
      txType: p.txType,
      bizNo: p.bizNo || null,
      fromAddress: p.fromAddress || 'SYSTEM',
      toAddress: p.toAddress || null,
      payloadHash,
      nonce,
    });
    const signature = p.privateKey ? cu.sign(p.privateKey, signContent) : null;

    return {
      txId,
      txType: p.txType,
      bizNo: p.bizNo || null,
      blockIndex: null,
      fromAddress: p.fromAddress || 'SYSTEM',
      toAddress: p.toAddress || null,
      payloadHash,
      payload: p.payload || {},
      signature,
      pubKey: p.publicKey || null,
      nonce,
      fee: p.fee || 0,
      status: 'PENDING',
      createdAt: Date.now(),
      signContent, // 仅内存中使用，供验签
    };
  }

  /** 内存中校验一笔交易的签名 */
  static isTxSignatureValid(tx) {
    if (!tx.signature || !tx.pubKey) return false;
```

```
    const content = cu.stableStringify({
      txId: tx.txId,
      txType: tx.txType,
      bizNo: tx.bizNo || null,
      fromAddress: tx.fromAddress,
      toAddress: tx.toAddress || null,
      payloadHash: tx.payloadHash,
      nonce: tx.nonce,
    });
    return cu.verify(tx.pubKey, content, tx.signature);
  }

  /**
   * 【第五道校验】链下重算绑定：用交易明文负载重算 SHA-256，与链上锚定摘要比对。
   * ------------------------------------------------------------------
   * 前四道校验（区块头哈希 / 前向引用 / Merkle 根 / 出块与交易签名）全部只覆盖
   * **链上字段**。它们能证明"链上记录没人改过"，却证明不了"链下明细和链上锚点还对得上"。
   * 若攻击者只改链下 payload 正文而不动 payloadHash，前四道全绿、篡改却已发生。
   * 本方法补上这个断点：hashObject(payload) 必须等于 payloadHash。
   * @param {object} tx 至少含 { payload, payloadHash }
   * @returns {boolean}
   */
  static isTxPayloadBound(tx) {
    if (!tx || !tx.payloadHash) return false;
    return cu.verifyPayloadBinding(tx.payload, tx.payloadHash);
  }

  /** 交易落库为 PENDING */
  async persistTx(tx) {
    await this.db.query(
      `INSERT INTO chain_tx
        (tx_id, tx_type, biz_no, block_index, from_address, to_address, payload_hash,
         payload_json, signature, pub_key, nonce, gas_fee, status, created_at)
       VALUES (?,?,?,NULL,?,?,?,?,?,?,?,?,'PENDING',NOW())
       ON DUPLICATE KEY UPDATE tx_id = tx_id`,
      [
        tx.txId, tx.txType, tx.bizNo, tx.fromAddress, tx.toAddress, tx.payloadHash,
        JSON.stringify(tx.payload), tx.signature, tx.pubKey, tx.nonce, tx.fee,
      ]
    );
  }

  /**
   * 批量落库为 PENDING（性能优化）
   * ------------------------------------------------------------------
   * 与 persistTx 写入完全相同的行，但用单条 INSERT ... VALUES (...),(...) 提交。
   * 用途：初始化脚本一次要灌 900+ 笔存证，逐条 await 在跨地域（如本机 → 云端 MySQL）
   * 场景下光是网络往返就要数分钟；批量后往返次数从 N 降到 N/50。
   * 单进程高频提交时同样受益。业务语义与逐条写入完全一致。
   */
```

```
async persistTxBatch(txs, chunk = 50) {
  if (!txs || !txs.length) return 0;
  const cols = ['tx_id', 'tx_type', 'biz_no', 'from_address', 'to_address', 'payload_hash',
    'payload_json', 'signature', 'pub_key', 'nonce', 'gas_fee'];
  let n = 0;
  for (let i = 0; i < txs.length; i += chunk) {
    const slice = txs.slice(i, i + chunk);
    const ph = slice.map(() => "(?,?,?,?,?,?,?,?,?,?,?,'PENDING',NOW())").join(',');
    const params = slice.flatMap((tx) => [
      tx.txId, tx.txType, tx.bizNo, tx.fromAddress, tx.toAddress, tx.payloadHash,
      JSON.stringify(tx.payload), tx.signature, tx.pubKey, tx.nonce, tx.fee,
    ]);
    await this.db.query(
      `INSERT INTO chain_tx (${cols.join(',')}, status, created_at)
       VALUES ${ph}
       ON DUPLICATE KEY UPDATE tx_id = tx_id`,
      params
    );
    n += slice.length;
  }
  return n;
}

/**
 * 提交交易：入池，并在满足批大小时自动出块
 * @returns {Promise<{tx:object, block:object|null}>}
 */
async submitTx(tx) {
  await this.persistTx(tx);
  this.pool.push(tx);
  if (this.autoMine && this.pool.length >= this.batchSize) {
    const block = await this.mineBlock();
    return { tx, block };
  }
  return { tx, block: null };
}

/* ================================================================
 * 出块
 * ================================================================ */

/**
 * 打包当前交易池并出块
 * @param {object} opts { miner, genesis, txs }
 * @returns {Promise<object>} 新区块
 */
async mineBlock(opts = {}) {
  if (this.mining) throw new Error('出块进行中，请稍后');
  this.mining = true;
  try {
```

```
    const genesis = !!opts.genesis;
    const txs = opts.txs || (genesis ? [] : this.pool.splice(0, this.pool.length));
    const prev = this.blocks[this.blocks.length - 1];
    const index = prev ? prev.index + 1 : 0;
    // 时间戳取整到秒（与 DATETIME 存储精度一致，保证重新装载后哈希可复现）
    const rawMs = opts.timestamp
      ? new Date(opts.timestamp).getTime()
      : (genesis ? Date.now() - 1000 * 60 * 60 * 24 : Date.now());
    const timestamp = Math.floor(rawMs / 1000);
    const root = cu.merkleRoot(txs.map((t) => t.txId));
    const prevHash = prev ? prev.hash : GENESIS_PREV_HASH;

    // 按共识类型出块：联盟链默认 PoA（授权节点轮值 + 签名），POW 为历史兼容路径
    const block = this.consensusMode === 'POA'
      ? await this._buildPoaBlock({ index, timestamp, root, prevHash, txs, genesis, opts })
      : this._buildPowBlock({ index, timestamp, root, prevHash, txs, genesis, opts });

    await this.db.query(
      `INSERT INTO chain_block
        (block_index, block_hash, prev_hash, merkle_root, nonce, difficulty, consensus,
         proposer, proposer_sig, tx_count, miner, size_bytes, mine_time, block_time, created_at)
       VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?,NOW())`,
      [
        block.index, block.hash, block.prevHash, block.merkleRoot, block.nonce,
        block.difficulty, block.consensus, block.proposer, block.proposerSig,
        block.txCount, block.miner, block.sizeBytes, block.mineTime,
        // 传字符串而不是 Date 对象：驱动不再做任何本地时区换算，
        // 配合会话 time_zone='+00:00'，写入值与读出秒数在 MySQL / TiDB 上完全一致。
        Blockchain.utcDateString(block.timestamp),
      ]
    );

    if (txs.length) {
      const ids = txs.map((t) => t.txId);
      await this.db.query(
        `UPDATE chain_tx SET block_index = ?, status = 'CONFIRMED', confirmed_at = NOW()
         WHERE tx_id IN (?)`,
        [block.index, ids]
      );
      txs.forEach((t) => { t.blockIndex = block.index; t.status = 'CONFIRMED'; });
    }

    // 联盟链：登记本轮出块节点的统计（出块数 + 最近出块时间）
    if (block.consensus === 'POA' && block.proposer) {
      try {
        await this.db.query(
          `UPDATE chain_node SET proposed_blocks = proposed_blocks + 1, last_propose_at = NOW()
           WHERE node_id = ?`,
          [block.proposer]
        );
```

```
      } catch (e) { /* 节点表不可用时忽略统计 */ }
    }

    this.blocks.push(block);
    this._bumpRev(); // 新区块入链 → 全链校验缓存失效
    return block;
  } finally {
    this.mining = false;
  }
}

/**
 * 构造 PoA 区块：由本轮被授权的联盟节点出块，并对区块头签名。
 * 无挖矿，出块耗时接近 0 —— 这正是联盟链相对公有链的性能优势。
 */
async _buildPoaBlock({ index, timestamp, root, prevHash, txs, genesis, opts }) {
  const proposer = await this.pickProposer(index, opts.proposer);
  const header = {
    index, timestamp, merkleRoot: root, prevHash,
    consensus: 'POA', proposer: proposer.nodeId, nonce: 0,
  };
  const hash = Blockchain.calcHash(header);
  const proposerSig = cu.sign(proposer.privateKey, Blockchain.signingPayload(header));
  return {
    index, timestamp, merkleRoot: root, prevHash,
    consensus: 'POA', proposer: proposer.nodeId, nonce: 0, difficulty: 0,
    hash, proposerSig,
    txCount: txs.length, miner: proposer.nodeId, mineTime: 0,
    sizeBytes: Buffer.byteLength(JSON.stringify({ header, txs: txs.map((t) => t.txId) })),
  };
}

/** 构造 PoW 区块（历史兼容路径，保留以便旧链校验与迁移） */
_buildPowBlock({ index, timestamp, root, prevHash, txs, genesis, opts }) {
  const header = { index, timestamp, merkleRoot: root, prevHash, difficulty: this.difficulty, nonce: 0 };
  const pow = Blockchain.proofOfWork(header);
  return {
    index, timestamp, merkleRoot: root, prevHash,
    consensus: 'POW', proposer: opts.miner || this.nodeName, nonce: pow.nonce,
    difficulty: this.difficulty, hash: pow.hash, proposerSig: null,
    txCount: txs.length, miner: opts.miner || this.nodeName, mineTime: pow.elapsed,
    sizeBytes: Buffer.byteLength(JSON.stringify({ header, txs: txs.map((t) => t.txId) })),
  };
}

/**
 * 选出本轮出块节点（PoA）：从 chain_node 白名单里取"状态 ACTIVE 且具备出块权"的节点，
 * 按 vote_weight 展开成轮值序列后取 index % seq.length —— 权重越高、出块机会越多。
 * 若链上尚未登记任何授权节点，退化为本节点单节点联盟（保证系统可启动）。
 */
```

```
async pickProposer(index, forced) {
  if (forced) return this._nodeWallet(forced);
  try {
    const [rows] = await this.db.query(
      "SELECT node_id, vote_weight FROM chain_node WHERE status = 'ACTIVE' AND is_authorized = 1 ORDER BY id"
    );
    if (rows.length) {
      const seq = [];
      rows.forEach((r) => {
        const w = Math.max(1, Number(r.vote_weight) || 1);
        for (let i = 0; i < w; i++) seq.push(r.node_id);
      });
      return this._nodeWallet(seq[index % seq.length]);
    }
  } catch (e) { /* chain_node 表不存在时退化为单节点 */ }
  return this._nodeWallet(this.nodeName);
}

/** 取某节点的钱包（确定性派生，label 形如 node:xxx），用于出块签名 */
_nodeWallet(nodeId) {
  const w = keystore.walletFor(`node:${nodeId}`);
  return { nodeId, address: w.address, publicKey: w.publicKey, privateKey: w.privateKey };
}

/* ================================================================
 * 链完整性校验
 * ================================================================ */

/**
 * 带短时缓存的全链校验 —— 面向「高频只读巡检」场景（浏览器/监管端轮询该接口）。
 *
 * 为什么需要缓存：validateChain 是 O(区块数 × 交易数) 的重活，一次调用要对
 * 每个区块做 1 次区块头哈希 + 1 次出块签名验签（ECDSA ≈ 0.5 ms）+ 1 次 Merkle 建根，
 * 还要对每笔交易做 1 次链下重算绑定。500+ 区块时单次耗时进入百毫秒级，
 * 并发巡检会把单进程 CPU 打满（实测线上 100 并发全部超时）。
 *
 * 安全前提：缓存键包含 `_rev`（链数据变更版本号）与链高。任何写入 ——
 * 出块、篡改、修复、重新装载 —— 都会 `_bumpRev()` 使缓存立即失效，
 * 因此**不存在「链已被破坏但缓存仍报 valid=true」的窗口**。
 *
 * @param {object} opts { fromIndex, ttlMs } ttlMs 默认 5000，传 0 可强制重算
 * @returns {Promise<object>} validateChain 的结果，额外带 cached / cacheAgeMs 字段
 */
async validateChainCached(opts = {}) {
  const from = Number(opts.fromIndex) || 0;
  const ttl = opts.ttlMs === undefined ? 5000 : Number(opts.ttlMs);
  const tip = this.blocks.length ? this.blocks[this.blocks.length - 1].index : -1;
  const key = `${from}:${tip}:${this.blocks.length}:${this._rev}`;
  const now = Date.now();
  const c = this._validateCache;
```

```
  if (ttl > 0 && c && c.key === key && (now - c.at) < ttl) {
    return { ...c.value, cached: true, cacheAgeMs: now - c.at };
  }
  const value = await this.validateChain({ fromIndex: from });
  this._validateCache = { key, at: Date.now(), value };
  return { ...value, cached: false, cacheAgeMs: 0 };
}

/**
 * 全链校验（五道校验，逐层递进）：
 *   ① 区块头哈希重算          —— 链上区块内容是否被改
 *   ② 出块签名 / PoW / 出块资格 —— 是否由获授权节点出块、区块头是否被改
 *   ③ 前向引用 prevHash       —— 链是否断裂、区块是否被抽换
 *   ④ Merkle 根重算           —— 区块内交易是否被增删
 *   ⑤ 链下重算绑定            —— 链下业务明文是否仍与链上锚定摘要一致
 * 前四道只覆盖链上字段；第五道是"链上存证 + 链下存储"架构下唯一的链下完整性出口。
 * @param {object} opts { fromIndex } 仅校验某高度之后
 * @returns {Promise<object>} { valid, checkedBlocks, height, coverage, errors }
 */
async validateChain(opts = {}) {
  await this.syncTimeZone();
  const from = opts.fromIndex || 0;
  const [blocks] = await this.db.query(
    'SELECT *, UNIX_TIMESTAMP(block_time) AS ts FROM chain_block WHERE block_index >= ? ORDER BY block_index ASC', [from]
  );
  const errors = [];
  let checked = 0;

  // 联盟链：加载授权节点公钥与状态，用于验证 PoA 出块签名与出块资格
  const nodePubKey = new Map();
  const nodeStatus = new Map();
  try {
    const [nodes] = await this.db.query('SELECT node_id, public_key, status, is_authorized FROM chain_node');
    nodes.forEach((n) => {
      nodePubKey.set(n.node_id, n.public_key);
      nodeStatus.set(n.node_id, { status: n.status, auth: Number(n.is_authorized) });
    });
  } catch (e) { /* 无节点表时退化为仅验哈希链 */ }

  /* 一次性把所有区块的交易取回来，按 block_index 分组，避免"每块一次查询"。
   * 原实现是 for 循环里逐块 SELECT，本地 MySQL 上无所谓（<1 ms），
   * 但部署到云端后数据库与 Web 服务跨地域，155 块 = 155 次网络往返，
   * 实测一次全链校验要跑几十分钟，接口必然超时。
   * 改成单条查询 + 内存分组后，往返次数恒为 1。 */
  const [allTx] = await this.db.query(
    `SELECT block_index, tx_id, payload_hash, payload_json
       FROM chain_tx WHERE block_index >= ? ORDER BY block_index ASC, id ASC`,
    [from]
  );
  const txByBlock = new Map();      // 区块高度 → [tx_id]（供 Merkle 根重算）
```

```
const txRowsByBlock = new Map();  // 区块高度 → [整行]（供第五道「链下重算绑定」校验）
for (const t of allTx) {
  const k = Number(t.block_index);
  if (!txByBlock.has(k)) txByBlock.set(k, []);
  txByBlock.get(k).push(t.tx_id);
  if (!txRowsByBlock.has(k)) txRowsByBlock.set(k, []);
  txRowsByBlock.get(k).push(t);
}

// 第五道校验的计数器与错误明细（明细限量，避免异常时错误数组爆掉接口响应）
let txChecked = 0;
let payloadBound = 0;
let payloadMismatch = 0;
const payloadErrors = [];
const PAYLOAD_ERROR_LIMIT = 20;

for (let i = 0; i < blocks.length; i++) {
  const raw = blocks[i];
  const consensus = String(raw.consensus || 'POW').toUpperCase();
  const b = {
    index: Number(raw.block_index),
    timestamp: Number(raw.ts),
    merkleRoot: raw.merkle_root,
    prevHash: raw.prev_hash,
    difficulty: Number(raw.difficulty),
    nonce: Number(raw.nonce),
    consensus,
    proposer: raw.proposer || null,
    hash: raw.block_hash,
  };
  checked++;

  // (1) 重算哈希
  const expect = Blockchain.calcHash(b);
  if (expect !== b.hash) {
    errors.push({
      index: b.index, type: 'HASH_MISMATCH',
      message: `区块 #${b.index} 内容哈希与记录不符：期望 ${expect.slice(0, 16)}…，实际 ${b.hash.slice(0, 16)}…`,
    });
  }

  // (2) 共识校验：PoA 验出块签名与授权资格；PoW 校验前导零
  if (consensus === 'POA') {
    const local = keystore.all()[`node:${b.proposer}`];
    const pub = nodePubKey.get(b.proposer) || (local ? local.publicKey : null);
    const sigValid = !!(raw.proposer_sig && pub
      && cu.verify(pub, Blockchain.signingPayload(b), raw.proposer_sig));
    if (!sigValid) {
      errors.push({
        index: b.index, type: 'POA_SIGNATURE_INVALID',
```

```
        message: `区块 #${b.index} 的 PoA 出块签名无效（出块节点 ${b.proposer} 签名校验失败）`,
      });
    }
    const st = nodeStatus.get(b.proposer);
    if (st && (st.status !== 'ACTIVE' || !st.auth)) {
      errors.push({
        index: b.index, type: 'PROPOSER_UNAUTHORIZED',
        message: `区块 #${b.index} 由未获授权的节点 ${b.proposer} 出块（当前状态 ${st.status}）`,
      });
    }
  } else if (!String(b.hash).startsWith(cu.ZERO.repeat(b.difficulty))) {
    errors.push({
      index: b.index, type: 'POW_INVALID',
      message: `区块 #${b.index} 不满足难度 ${b.difficulty} 的工作量证明`,
    });
  }

  // (3) 前向引用
  if (i > 0) {
    const prevRaw = blocks[i - 1];
    if (b.prevHash !== prevRaw.block_hash) {
      errors.push({
        index: b.index, type: 'PREV_HASH_MISMATCH',
        message: `区块 #${b.index} 的 prevHash 未指向区块 #${b.index - 1}（链断裂）`,
      });
    }
  }

  // (4) Merkle 根与交易表比对（交易已在上方一次性取回）
  const txRows = txByBlock.get(b.index) || [];
  const root = cu.merkleRoot(txRows);
  if (root !== b.merkleRoot) {
    errors.push({
      index: b.index, type: 'MERKLE_ROOT_MISMATCH',
      message: `区块 #${b.index} 的交易 Merkle 根与链上交易表不一致（交易被增删改）`,
    });
  }

  // (5) 链下重算绑定：逐笔用明文负载重算摘要，与链上锚定摘要比对。
  //     这是唯一能发现"链下业务明细被改、链上锚点未变"的校验点。
  const bindRows = txRowsByBlock.get(b.index) || [];
  for (const t of bindRows) {
    txChecked++;
    const payload = t.payload_json === null || t.payload_json === undefined
      ? null : safeParse(t.payload_json);
    if (cu.verifyPayloadBinding(payload, t.payload_hash)) {
      payloadBound++;
    } else {
      payloadMismatch++;
      if (payloadErrors.length < PAYLOAD_ERROR_LIMIT) {
```

```
          payloadErrors.push({
            index: b.index, txId: t.tx_id, type: 'PAYLOAD_BINDING_MISMATCH',
            message: `区块 #${b.index} 的交易 ${String(t.tx_id).slice(0, 16)}… 链下明细与链上锚定摘要不符`
              + '（链下业务数据被改动，或锚点被改动）',
          });
        }
      }
    }
  }

  // 把第五道校验的失配汇总进 errors（限量明细 + 一条汇总）
  for (const e of payloadErrors) errors.push(e);
  if (payloadMismatch > PAYLOAD_ERROR_LIMIT) {
    errors.push({
      index: -1, type: 'PAYLOAD_BINDING_MISMATCH_SUMMARY',
      message: `另有 ${payloadMismatch - PAYLOAD_ERROR_LIMIT} 笔交易存在链下重算绑定失配（明细已省略）`,
    });
  }

  return {
    valid: errors.length === 0,
    checkedBlocks: checked,
    height: this.blocks.length ? this.blocks[this.blocks.length - 1].index : -1,
    /* 校验覆盖面（透明度指标）：本次到底验了什么、验了多少 */
    coverage: {
      blocksHashRecomputed: checked,
      merkleRootsRecomputed: checked,
      txPayloadBindingChecked: txChecked,
      txPayloadBoundOk: payloadBound,
      txPayloadBindingMismatch: payloadMismatch,
    },
    errors,
    verifiedAt: new Date().toISOString(),
  };
}

/* ================================================================
 * 篡改演示 / 修复
 * ================================================================ */

/**
 * 【演示】模拟一个恶意节点绕过应用层、直接修改数据库中的区块数据。
 * 修改后不重算哈希 —— 这正是攻击者最容易犯的错，也是区块链能抓到的点。
 *
 * 支持两类篡改目标：
 *   · 链上字段（merkle_root / prev_hash / nonce / block_time）—— 由前四道校验捕获
 *   · 链下明细（offchain_payload）—— 只改 chain_tx.payload_json 正文、**不动**
 *     payload_hash 与区块头。这条路径前四道校验全部通过，只有第五道「链下重算绑定」
 *     能抓到，用于演示"链上没被改 ≠ 数据没被改"。
 *
```

```
   * @param {number} blockIndex
   * @param {string} field 'merkle_root' | 'prev_hash' | 'nonce' | 'block_time' | 'offchain_payload'
   * @returns {Promise<object>} 篡改前后对比
   */
  async tamper(blockIndex, field = 'merkle_root') {
    if (field === 'offchain_payload') return this.tamperOffChainPayload(blockIndex);

    const [rows] = await this.db.query(
      'SELECT * FROM chain_block WHERE block_index = ?', [blockIndex]
    );
    if (!rows.length) throw new Error(`区块 #${blockIndex} 不存在`);
    const before = rows[0];
    const col = field === 'prev_hash' ? 'prev_hash'
      : field === 'nonce' ? 'nonce'
        : field === 'block_time' ? 'block_time' : 'merkle_root';

    let newValue;
    if (col === 'nonce') {
      newValue = Number(before.nonce) + 1;
    } else if (col === 'block_time') {
      newValue = new Date(new Date(before.block_time).getTime() - 86400000);
    } else {
      // 翻转哈希的最后一个字符
      newValue = String(before[col]).slice(0, -1) + (String(before[col]).endsWith('a') ? 'b' : 'a');
    }

    await this.db.query(
      `UPDATE chain_block SET ${col} = ? WHERE block_index = ?`, [newValue, blockIndex]
    );
    this._bumpRev(); // 链上字段被改写 → 缓存必须失效，否则会把已破坏的链误报为 valid

    const validation = await this.validateChain({ fromIndex: 0 });
    return {
      blockIndex,
      field: col,
      before: before[col],
      after: newValue,
      validation,
      tip: '链上数据一旦被改写，全链校验立刻失败 —— 这就是"不可篡改"的技术含义。',
    };
  }

  /**
   * 【演示】只篡改**链下明细**：改动某区块内一笔交易的 payload_json 正文，
   * 保持 payload_hash、tx_id、区块头、Merkle 根全部不变。
   * 用于证明第五道「链下重算绑定」校验的必要性。
   *
   * 演示前先把原文备份到 chain_payload_backup（幂等建表），
   * 使 `repairOffChainPayload()` 可一键还原，演示可反复进行、不污染数据。
   * @param {number} blockIndex
```

```
   * @returns {Promise<object>}
   */
  async tamperOffChainPayload(blockIndex) {
    const [rows] = await this.db.query(
      `SELECT tx_id, payload_hash, payload_json FROM chain_tx
        WHERE block_index = ? ORDER BY id ASC LIMIT 1`, [blockIndex]
    );
    if (!rows.length) throw new Error(`区块 #${blockIndex} 内没有交易，无法演示链下明细篡改`);
    const tx = rows[0];

    // 先把原文备份走（同一 tx 只保留最早一次原文，避免重复演示时把被污染的值当原文）
    await this.db.query(
      `CREATE TABLE IF NOT EXISTS chain_payload_backup (
         tx_id        VARCHAR(80) NOT NULL,
         payload_json TEXT                DEFAULT NULL,
         backed_at    DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
         PRIMARY KEY (tx_id)
       ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci`
    );
    await this.db.query(
      `INSERT IGNORE INTO chain_payload_backup (tx_id, payload_json, backed_at)
       VALUES (?, ?, NOW())`,
      [tx.tx_id, tx.payload_json]
    );

    const payload = safeParse(tx.payload_json);
    if (!payload || typeof payload !== 'object') {
      throw new Error('该交易的 payload_json 不是可解析的对象，无法演示');
    }

    // 找第一个数值字段并把它的值改掉（不改 key，保持结构不变）
    const key = Object.keys(payload).find((k) => typeof payload[k] === 'number');
    const beforeJson = JSON.stringify(payload);
    if (key) {
      payload[key] = Math.round((Number(payload[key]) + 1234.5) * 100) / 100;
    } else {
      payload.__tampered = true;  // 没有数值字段就注入一个标记字段
    }
    const afterJson = JSON.stringify(payload);

    await this.db.query(
      'UPDATE chain_tx SET payload_json = ? WHERE tx_id = ?', [afterJson, tx.tx_id]
    );
    this._bumpRev(); // 链下明细被改写 → 缓存失效（第五道校验的结果随之变化）

    const validation = await this.validateChain({ fromIndex: 0 });
    return {
      blockIndex,
      field: 'offchain_payload',
      txId: tx.tx_id,
```

```
      changedKey: key || '__tampered',
      before: beforeJson.slice(0, 160),
      after: afterJson.slice(0, 160),
      payloadHashUnchanged: tx.payload_hash,
      backups: 1,
      note: '注意：payload_hash、tx_id、区块头、Merkle 根**全部未改动**。'
        + '前四道链上校验因此全部通过（链上确实没人动过），'
        + '只有第五道「链下重算绑定」能发现链下明细已与链上锚点脱钩。',
      validation,
      tip: '链上存证必须配套链下重算 —— 否则"链上没被改"会被误读为"数据没被改"。',
    };
  }

  /**
   * 把链下 payload_json 从备份还原（配合 tamperOffChainPayload 使用）。
   * 不传 blockIndex 时还原全部备份。
   * @param {number} [blockIndex]
   * @returns {Promise<object>}
   */
  async repairOffChainPayload(blockIndex) {
    const scoped = Number.isFinite(Number(blockIndex));
    // 注意：不要在 SQL 里 JOIN chain_tx —— 演示表是运行期按需创建的，
    // 其列排序规则取自服务器默认（MySQL 8 为 utf8mb4_0900_ai_ci），
    // 而既有 chain_tx 为 utf8mb4_general_ci，跨排序规则 JOIN VARCHAR 会报
    // ER_CANT_AGGREGATE_2COLLATIONS（本机实测踩到）。改为两次独立查询 + 内存求交，
    // 既绕开排序规则差异，也避免各版本 MySQL/TiDB 的行为分歧。
    let backups;
    try {
      [backups] = await this.db.query(
        'SELECT tx_id, payload_json AS backup_json FROM chain_payload_backup'
      );
    } catch (e) {
      // 备份表不存在 → 没有需要还原的内容
      return { restored: 0, validation: null, tip: '没有链下明细篡改记录需要还原。' };
    }
    if (!backups.length) return { restored: 0, validation: null, tip: '没有需要还原的内容。' };

    let allow = null;
    if (scoped) {
      const [txRows] = await this.db.query(
        'SELECT tx_id FROM chain_tx WHERE block_index = ?', [Number(blockIndex)]
      );
      allow = new Set(txRows.map((r) => r.tx_id));
    }

    let restored = 0;
    for (const r of backups) {
      if (allow && !allow.has(r.tx_id)) continue;
      await this.db.query('UPDATE chain_tx SET payload_json = ? WHERE tx_id = ?', [r.backup_json, r.tx_id]);
      await this.db.query('DELETE FROM chain_payload_backup WHERE tx_id = ?', [r.tx_id]);
```

```
      restored++;
    }
    this._bumpRev(); // 链下明细还原 → 缓存失效

    const validation = await this.validateChain({ fromIndex: 0 });
    return {
      restored,
      validation,
      tip: restored ? '链下明细已还原为备份原文，第五道校验重新通过。' : '没有匹配到需要还原的内容。',
    };
  }

  /**
   * 修复链：从指定高度起重算哈希并重做 PoW（模拟攻击者想"续上"链所需要付出的代价）。
   * 演示用：改一个块，后面所有块都得重挖，且其它节点只要比对过旧哈希就能发现。
   * @param {number} fromIndex
   * @returns {Promise<object>}
   */
  async repairChain(fromIndex) {
    await this.syncTimeZone();
    const [rows] = await this.db.query(
      'SELECT *, UNIX_TIMESTAMP(block_time) AS ts FROM chain_block WHERE block_index >= ? ORDER BY block_index ASC', [fromIr
    );
    const repaired = [];
    let prevHash = null;
    if (fromIndex > 0) {
      const [p] = await this.db.query(
        'SELECT block_hash FROM chain_block WHERE block_index = ?', [fromIndex - 1]
      );
      prevHash = p.length ? p[0].block_hash : GENESIS_PREV_HASH;
    }
    // 同 validateChain：交易一次性取回并按块分组，避免逐块查询造成的 N 次网络往返
    const [allTx] = await this.db.query(
      'SELECT block_index, tx_id FROM chain_tx WHERE block_index >= ? ORDER BY block_index ASC, id ASC',
      [fromIndex]
    );
    const txByBlock = new Map();
    for (const t of allTx) {
      const k = Number(t.block_index);
      if (!txByBlock.has(k)) txByBlock.set(k, []);
      txByBlock.get(k).push(t.tx_id);
    }
    for (const raw of rows) {
      const txRows = txByBlock.get(Number(raw.block_index)) || [];
      const merkle = cu.merkleRoot(txRows);
      const consensus = String(raw.consensus || 'POW').toUpperCase();
      const proposer = raw.proposer || raw.miner || this.nodeName;
      const header = {
        index: Number(raw.block_index),
        timestamp: Number(raw.ts),
```

```
      merkleRoot: merkle,
      prevHash: prevHash || raw.prev_hash,
      difficulty: Number(raw.difficulty) || this.difficulty,
      nonce: Number(raw.nonce) || 0,
      consensus,
      proposer,
    };
    let newHash;
    let sig = raw.proposer_sig || null;
    let cost = 0;
    if (consensus === 'POA') {
      // 联盟链：修复 = 用该块出块节点的私钥重新签名（无挖矿成本，因此"篡改收益"依赖准入治理而非算力）
      newHash = Blockchain.calcHash(header);
      const w = keystore.walletFor(`node:${proposer}`);
      sig = cu.sign(w.privateKey, Blockchain.signingPayload(header));
      header.nonce = 0;
    } else {
      const pow = Blockchain.proofOfWork({ ...header, nonce: 0 });
      newHash = pow.hash;
      header.nonce = pow.nonce;
      sig = null;
      cost = pow.elapsed;
    }
    await this.db.query(
      `UPDATE chain_block SET block_hash=?, prev_hash=?, merkle_root=?, nonce=?, proposer_sig=?, mine_time=?
       WHERE block_index = ?`,
      [newHash, header.prevHash, merkle, header.nonce, sig, cost, raw.block_index]
    );
    repaired.push({ index: header.index, newHash, cost });
    prevHash = newHash;
  }
  await this.init();
  const validation = await this.validateChain();
  return { fromIndex, repaired, validation };
}

/* ================================================================
 * 统计
 * ================================================================ */

async stats() {
  const [[blk]] = await this.db.query(
    `SELECT COUNT(*) AS total_blocks, IFNULL(SUM(tx_count),0) AS total_txs,
            IFNULL(AVG(mine_time),0) AS avg_mine_time, IFNULL(MAX(difficulty),0) AS difficulty,
            IFNULL(SUM(size_bytes),0) AS total_size
     FROM chain_block`
  );
  const [[tx]] = await this.db.query(
    `SELECT COUNT(*) AS total, SUM(status='CONFIRMED') AS confirmed, SUM(status='PENDING') AS pending
     FROM chain_tx`
```

```
    );
    const [byType] = await this.db.query(
      'SELECT tx_type, COUNT(*) AS cnt FROM chain_tx GROUP BY tx_type ORDER BY cnt DESC'
    );
    const [recent] = await this.db.query(
      `SELECT DATE_FORMAT(block_time,'%Y-%m-%d %H:00') AS hour, COUNT(*) AS cnt
       FROM chain_block GROUP BY hour ORDER BY hour DESC LIMIT 24`
    );
    const tip = this.blocks[this.blocks.length - 1] || null;
    let activeNodes = 0;
    let poaBlocks = 0;
    try {
      const [[nc]] = await this.db.query("SELECT COUNT(*) AS c FROM chain_node WHERE status = 'ACTIVE'");
      activeNodes = Number(nc.c) || 0;
      const [[pb]] = await this.db.query("SELECT COUNT(*) AS c FROM chain_block WHERE consensus = 'POA'");
      poaBlocks = Number(pb.c) || 0;
    } catch (e) { /* 非联盟表结构时忽略 */ }
    return {
      height: tip ? tip.index : -1,
      tipHash: tip ? tip.hash : null,
      totalBlocks: Number(blk.total_blocks),
      totalTxs: Number(blk.total_txs),
      avgMineTime: Math.round(Number(blk.avg_mine_time)),
      difficulty: Number(blk.difficulty),
      totalSize: Number(blk.total_size),
      confirmed: Number(tx.confirmed || 0),
      pending: Number(tx.pending || 0),
      txByType: byType,
      blockRate: recent.reverse(),
      node: this.nodeName,
      consensus: this.consensusMode === 'POA'
        ? 'PoA（权威证明）+ 联盟节点轮值出块'
        : 'PoW（工作量证明）+ 最长链原则',
      consensusMode: this.consensusMode,
      activeNodes,
      poaBlocks,
      crypto: 'SHA-256 / secp256k1-ECDSA / Merkle Tree',
    };
  }
}

function safeParse(s) {
  try {
    // mysql2 对 TEXT 列返回字符串；若某部署把该列建成 JSON 类型，驱动可能已解析成对象，
    // 此时直接返回，避免对对象再 JSON.parse 触发异常。
    if (s !== null && typeof s === 'object') return s;
    return JSON.parse(s);
  } catch (e) { return null; }
}
```

```
module.exports = { Blockchain, GENESIS_PREV_HASH };

/* ================================================================
 * 文件：server/blockchain/crypto-utils.js
 * ================================================================ */
/**
 * crypto-utils.js —— 密码学工具层
 * ----------------------------------------------------------------
 * 本模块为整个"碳链通"平台提供真实的密码学原语，不依赖任何第三方区块链
 * 框架，全部基于 Node.js 内置 crypto 模块实现：
 *
 *   1. SHA-256 摘要          —— 区块哈希、交易哈希、Merkle 树、口令摘要
 *   2. Merkle 树             —— 区块内交易的完整性根，O(log n) 验证
 *   3. secp256k1 密钥对      —— 企业/机构的链上钱包身份
 *   4. ECDSA 数字签名        —— 交易不可抵赖（谁发的、发的是不是原文）
 *   5. 钱包地址派生          —— 由公钥单向派生 0x 地址，不可反推
 *
 * @module blockchain/crypto-utils
 */
'use strict';

const crypto = require('crypto');

/** 难度前缀字符：PoW 要求哈希以 N 个 '0' 开头 */
const ZERO = '0';

/* ================================================================
 * 一、摘要相关
 * ================================================================ */

/**
 * 计算 SHA-256 摘要（十六进制小写）
 * @param {string|Buffer} data 待摘要数据
 * @returns {string} 64 位十六进制字符串
 */
function sha256(data) {
  return crypto.createHash('sha256').update(data).digest('hex');
}

/**
 * 对任意对象做"规范化序列化"后再摘要。
 * 关键点：key 必须排序，否则 {a:1,b:2} 与 {b:2,a:1} 会算出不同哈希，
 * 同一个业务数据在不同节点上就得不到一致的摘要，共识会崩。
 * @param {*} obj
 * @returns {string}
 */
function stableStringify(obj) {
  if (obj === null || obj === undefined) return 'null';
  if (typeof obj !== 'object') return JSON.stringify(obj);
  if (Array.isArray(obj)) return '[' + obj.map(stableStringify).join(',') + ']';
```

```
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

function httpJson(url) {
  return new Promise((resolve, reject) => {
    const req = http.get(url, (res) => {
      let d = '';
      res.on('data', (c) => { d += c; });
      res.on('end', () => { try { resolve(JSON.parse(d)); } catch (e) { reject(new Error('bad json: ' + d.slice(0, 200))); }
    });
    req.on('error', reject);
    req.setTimeout(8000, () => req.destroy(new Error('timeout ' + url)));
  });
}

async function api(pathname, method = 'GET', body, token) {
  const headers = { 'Content-Type': 'application/json' };
  if (token) headers.Authorization = 'Bearer ' + token;
  const res = await fetch(BASE + '/api' + pathname, {
    method, headers, body: body ? JSON.stringify(body) : undefined,
  });
  const json = await res.json();
  if (json.code !== 0) throw new Error(pathname + ' -> ' + json.message);
  return json.data;
}

/* ---------------------------------------------------------------- CDP */
class CDP {
  constructor(ws) {
    this.ws = ws;
    this.seq = 0;
    this.pending = new Map();
    this.events = [];
    ws.addEventListener('message', (ev) => {
      let msg; try { msg = JSON.parse(ev.data); } catch (e) { return; }
      if (msg.id && this.pending.has(msg.id)) {
        const { resolve, reject } = this.pending.get(msg.id);
        this.pending.delete(msg.id);
        if (msg.error) reject(new Error(msg.method + ' ' + msg.error.message));
        else resolve(msg.result);
      } else if (msg.method) {
        this.events.push(msg);
      }
    });
  }
  send(method, params = {}) {
    const id = ++this.seq;
    this.ws.send(JSON.stringify({ id, method, params }));
    return new Promise((resolve, reject) => {
      this.pending.set(id, { resolve, reject });
      setTimeout(() => { if (this.pending.has(id)) { this.pending.delete(id); reject(new Error('cdp timeout: ' + method)); }
```

```
    });
  }
  async evaluate(expression, awaitPromise = true) {
    const r = await this.send('Runtime.evaluate', {
      expression, returnByValue: true, awaitPromise, userGesture: true,
    });
    if (r.exceptionDetails) {
      throw new Error('eval error: ' + (r.exceptionDetails.exception && r.exceptionDetails.exception.description || r.except
    }
    return r.result ? r.result.value : undefined;
  }
}

async function launchEdge() {
  // 清掉上一次残留：占用调试端口的旧实例会让新的 Edge 起不来（同一 profile 被锁）
  try {
    const busy = await new Promise((resolve) => {
      const ps = spawn('powershell', ['-NoProfile', '-Command',
        `(Get-NetTCPConnection -LocalPort ${PORT} -State Listen -ErrorAction SilentlyContinue).OwningProcess`], { stdio: ['i
      let out = '';
      ps.stdout.on('data', (d) => { out += d; });
      ps.on('close', () => resolve(out.trim()));
    });
    for (const pid of busy.split(/\s+/).filter(Boolean)) {
      spawn('taskkill', ['/PID', pid, '/T', '/F'], { stdio: 'ignore' });
      await sleep(700);
    }
  } catch (e) { /* 忽略 */ }

  const profile = path.join(TMP_ROOT, 'profile-' + Date.now());
  fs.mkdirSync(profile, { recursive: true });
  ACTIVE_PROFILE = profile;
  const args = [
    '--headless=new',
    '--remote-debugging-port=' + PORT,
    '--user-data-dir=' + profile,
    '--no-first-run', '--no-default-browser-check', '--disable-extensions',
    '--disable-gpu', '--hide-scrollbars', '--mute-audio',
    '--no-proxy-server', '--proxy-bypass-list=*',
    '--window-size=1680,1050',
    '--disable-features=Translate,BackForwardCache',
    'about:blank',
  ];
  if (HEADED) args.splice(0, 1);
  const child = spawn(EDGE, args, { detached: false, stdio: 'ignore' });
  child.unref();
  return { child, profile };
}

/** 结束后整棵进程树收干净，否则残留的 Edge 会拖垮下一次巡检 */
```

```
async function killEdge(child, profile) {
  try { if (child && child.pid) spawn('taskkill', ['/PID', String(child.pid), '/T', '/F'], { stdio: 'ignore' }); } catch (e)
  await sleep(800);
  /* 必须 await 到真正删掉为止：以前用 setTimeout 延迟删除，紧接着 process.exit()
     会把定时器一起掐掉，导致 profile 目录（每个约 40~50MB）永久堆在磁盘上。 */
  for (let i = 0; i < 3; i++) {
    try { fs.rmSync(profile, { recursive: true, force: true }); } catch (e) { /* ignore */ }
    if (!fs.existsSync(profile)) break;
    await sleep(700);
  }
  if (ACTIVE_PROFILE === profile) ACTIVE_PROFILE = null;
}

async function waitForTarget(timeoutMs = 25000) {
  const t0 = Date.now();
  while (Date.now() - t0 < timeoutMs) {
    try {
      const list = await httpJson(`http://127.0.0.1:${PORT}/json/list`);
      const page = list.find((t) => t.type === 'page' && t.webSocketDebuggerUrl);
      if (page) return page;
    } catch (e) { /* 端口未就绪 */ }
    await sleep(300);
  }
  throw new Error('未能在超时内连接到 Edge 调试端口 ' + PORT);
}

/* ---------------------------------------------------------------- 初始化脚本 */
const INIT_SCRIPT = `
window.__logs = [];
(function () {
  var oe = console.error, ow = console.warn;
  console.error = function () { window.__logs.push({ lvl: 'error', msg: Array.prototype.map.call(arguments, String).join(' '
  console.warn = function () { window.__logs.push({ lvl: 'warn', msg: Array.prototype.map.call(arguments, String).join(' ')
  window.addEventListener('error', function (e) {
    var t = e.target && e.target.tagName;
    if (t && (t === 'IMG' || t === 'SCRIPT' || t === 'LINK')) {
      window.__logs.push({ lvl: 'resource', msg: t + ' 加载失败: ' + (e.target.src || e.target.href) });
    } else {
      window.__logs.push({ lvl: 'uncaught', msg: String(e.message) + ' @ ' + (e.filename || '') + ':' + (e.lineno || 0) });
    }
  }, true);
  window.addEventListener('unhandledrejection', function (e) {
    var r = e.reason;
    window.__logs.push({ lvl: 'unhandledRejection', msg: String((r && r.message) || r) });
  });
})();
`;

const PROBE = `(function () {
  var app = document.getElementById('app');
```

```
  if (!app) return { fatal: 'no #app' };
  var txt = (app.innerText || '').replace(/\\s+/g, ' ').trim();
  var log = (window.__logs || []).filter(function (x) {
    return x.lvl === 'uncaught' || x.lvl === 'unhandledRejection' || x.lvl === 'error' || x.lvl === 'resource';
  });
  var warns = (window.__logs || []).filter(function (x) { return x.lvl === 'warn'; }).slice(0, 6);
  var chartCanvas = document.querySelectorAll('canvas').length;
  return {
    hash: location.hash,
    booting: !!document.querySelector('.boot'),
    htmlLen: app.innerHTML.length,
    textLen: txt.length,
    head: txt.slice(0, 110),
    cards: document.querySelectorAll('.card,.stat,.panel').length,
    tables: document.querySelectorAll('table').length,
    rows: document.querySelectorAll('tbody tr').length,
    charts: chartCanvas,
    errorBars: Array.prototype.map.call(document.querySelectorAll('.alertbar.err,.empty.err'), function (n) { return (n.inne
    warns: warns,
    unresolved: Array.prototype.filter.call(document.querySelectorAll('*'), function (n) {
      return n.tagName && n.tagName.indexOf('X-') === 0;
    }).map(function (n) { return n.tagName.toLowerCase(); }).filter(function (v, i, a) { return a.indexOf(v) === i; }),
    logs: log.slice(0, 12),
  };
})()`;

/* ---------------------------------------------------------------- 场景定义 */
const ROLE_ROUTES = {
  ENTERPRISE: [
    ['/ent/overview', '企业总览'],
    ['/ent/reports', '碳排放上报'],
    ['/ent/dynamic', '动态核算'],
    ['/ent/credit', '碳信用与抵销'],
    ['/ent/quota', '碳配额账户'],
    ['/ent/trade', '碳配额交易'],
    ['/ent/analytics', '数据分析'],
    ['/ent/wallet', '链上身份'],
    ['/ent/notices', '通知公告'],
  ],
  VERIFIER: [
    ['/ver/overview', '核查总览'],
    ['/ver/tasks', '核查任务'],
    ['/ver/reports', '核查报告'],
    ['/ver/wallet', '机构链上身份'],
    ['/ver/notices', '通知公告'],
  ],
  REGULATOR: [
    ['/reg/overview', '监管总览'],
    ['/reg/enterprises', '企业名录'],
    ['/reg/allocations', '配额分配'],
```

```
    ['/reg/trades', '交易监管'],
    ['/reg/dynamic', '动态核算监管'],
    ['/reg/credit', '碳资产总览'],
    ['/reg/alerts', '预警处置'],
    ['/reg/stats', '统计分析'],
    ['/reg/logs', '操作日志'],
    ['/reg/consortium', '联盟链治理'],
    ['/reg/system', '系统设置'],
    ['/reg/notices', '通知公告'],
  ],
};

const PUBLIC_ROUTES = [
  ['/login', '统一登录'],
  ['/screen', '数据公示大屏'],
  ['/explorer', '区块链浏览器'],
  ['/verify', '科研验证中心'],
  ['/cems', 'CEMS 实测流监控'],
];

/* ---------------------------------------------------------------- 主流程 */
async function main() {
  fs.mkdirSync(SHOT, { recursive: true });

  // 1) 取三端 token + 若干动态 ID
  const accounts = await api('/auth/demo-accounts');
  const tokens = {};
  const users = {};
  for (const a of accounts) {
    if (tokens[a.role]) continue;
    const d = await api('/auth/login', 'POST', { username: a.username, password: a.password });
    tokens[a.role] = d.token;
    users[a.role] = d.user;
  }
  const notices = await api('/public/notices?page=1&size=1', 'GET', null, null).catch(() => null);
  const noticeId = notices && notices.rows && notices.rows.length ? notices.rows[0].id : null;
  const chain = await api('/chain/stats');

  console.log(`[i] 令牌就绪: ${Object.keys(tokens).join(', ')}  链高度=${chain.height}`);

  // 2) 启动浏览器
  const { child: proc, profile } = await launchEdge();
  const target = await waitForTarget();
  const ws = new WebSocket(target.webSocketDebuggerUrl);
  await new Promise((resolve, reject) => {
    ws.addEventListener('open', resolve);
    ws.addEventListener('error', (e) => reject(new Error('ws error')));
  });
  const cdp = new CDP(ws);
```

```
await cdp.send('Page.enable');
await cdp.send('Runtime.enable');
await cdp.send('Log.enable');
await cdp.send('Page.addScriptToEvaluateOnNewDocument', { source: INIT_SCRIPT });
await cdp.send('Emulation.setDeviceMetricsOverride', {
  width: 1680, height: 1050, deviceScaleFactor: 1, mobile: false,
});

// about:blank 下 localStorage 不可用，先落到真实 origin
await cdp.send('Page.navigate', { url: `${BASE}/?_boot=1#/login` });
for (let i = 0; i < 60; i++) {
  await sleep(250);
  try {
    const u = await cdp.evaluate('location.origin');
    if (u && u.indexOf('127.0.0.1') > -1) break;
  } catch (e) { /* 导航中 */ }
}
await sleep(600);

const results = [];

/** 载入指定角色会话 + 路由（整页重载，确保 App.Store 与 localStorage 一致） */
async function loadRoute(role, route, { keepSession = false } = {}) {
  if (!keepSession) {
    const token = tokens[role] || '';
    const user = users[role] || null;
    await cdp.evaluate(`(function(){
      ${token ? `localStorage.setItem('cch_token', ${JSON.stringify(token)});` : `localStorage.removeItem('cch_token');`}
      ${user ? `localStorage.setItem('cch_user', ${JSON.stringify(JSON.stringify(user))});` : `localStorage.removeItem('cc
      return true;
    })()`);
  }
  const url = `${BASE}/?_=${Date.now()}#${route}`;
  await cdp.send('Page.navigate', { url });
  // 等待渲染稳定
  const t0 = Date.now();
  let probe = null;
  while (Date.now() - t0 < 14000) {
    await sleep(260);
    try { probe = await cdp.evaluate(PROBE); } catch (e) { continue; }
    if (probe && !probe.booting && probe.textLen > 60) break;
  }
  await sleep(1250); // 等异步数据与图表
  return probe;
}

async function shoot(name) {
  const metrics = await cdp.send('Page.getLayoutMetrics');
  const size = metrics.cssContentSize || metrics.contentSize;
  const h = Math.min(Math.max(size.height, 900), 5200);
```

```
  const r = await cdp.send('Page.captureScreenshot', {
    format: 'png', captureBeyondViewport: true,
    clip: { x: 0, y: 0, width: Math.max(size.width, 1680), height: h, scale: 1 },
  });
  const file = path.join(SHOT, name + '.png');
  fs.writeFileSync(file, Buffer.from(r.data, 'base64'));
  return { file: path.relative(ROOT, file), size: fs.statSync(file).size, height: h };
}

async function visit(role, route, label, seq) {
  const probe = await loadRoute(role, route);
  const shot = await shoot(`${String(seq).padStart(2, '0')}-${label}`);
  const pass = probe && !probe.booting && probe.textLen > 60
    && probe.logs.length === 0 && probe.unresolved.length === 0 && probe.errorBars.length === 0;
  results.push({ seq, role, route, label, pass, probe, shot });
  console.log(`${pass ? 'PASS' : 'FAIL'}  [${role || 'PUBLIC'}] ${route}  texts=${probe ? probe.textLen : '-'} charts=${pr
  return probe;
}

/* ---- A. 公开路由 ---- */
let seq = 1;
for (const [route, label] of PUBLIC_ROUTES) {
  await visit(null, route, label, seq++);
}
if (noticeId) await visit(null, `/notice/${noticeId}`, '公告详情', seq++);
await visit(null, '/explorer/block/1', '区块详情', seq++);
const tx = await api('/chain/txs?page=1&size=200').catch(() => null);
let txWithPath = null;
if (tx && tx.rows && tx.rows.length) {
  // 优先挑一笔「所在区块有多笔交易」的存证，这样交易详情里能展示完整 Merkle 证明路径
  for (const row of tx.rows.slice(0, 40)) {
    try {
      const d = await api('/chain/tx/' + row.tx_id);
      if (d.merkle && (d.merkle.path || []).length > 0) { txWithPath = row.tx_id; break; }
    } catch (e) { /* 跳过 */ }
  }
  if (!txWithPath) txWithPath = tx.rows[0].tx_id;
}
if (txWithPath) await visit(null, `/explorer/tx/${txWithPath}`, '交易详情（含 Merkle 路径）', seq++);

/* ---- B. 三端业务路由 ---- */
for (const role of ['ENTERPRISE', 'VERIFIER', 'REGULATOR']) {
  for (const [route, label] of ROLE_ROUTES[role]) {
    await visit(role, route, `${role.slice(0, 3)}-${label}`, seq++);
  }
}

/* ---- C. 交互回归 ---- */
const interactions = [];
```

```
// C1. 登录页真实登录（清空会话 → 填表 → 点登录）
await cdp.evaluate(`localStorage.clear(); true`);
await loadRoute(null, '/login', { keepSession: true });
const loginOk = await cdp.evaluate(`(function () {
  var inputs = document.querySelectorAll('.login-card input');
  if (inputs.length < 2) return { ok: false, why: 'no inputs' };
  var setter = Object.getOwnPropertyDescriptor(window.HTMLInputElement.prototype, 'value').set;
  setter.call(inputs[0], 'ent001'); inputs[0].dispatchEvent(new Event('input', { bubbles: true }));
  setter.call(inputs[1], '123456'); inputs[1].dispatchEvent(new Event('input', { bubbles: true }));
  var btn = document.querySelector('.login-card .btn.primary');
  if (!btn) return { ok: false, why: 'no button' };
  btn.click();
  return { ok: true };
})()`);
await sleep(2200);
const afterLogin = await cdp.evaluate(`({ hash: location.hash, hasToken: !!localStorage.getItem('cch_token') })`);
interactions.push({
  name: '登录页表单登录 → 跳转企业端',
  pass: loginOk.ok && /^#\/ent\//.test(afterLogin.hash) && afterLogin.hasToken,
  detail: JSON.stringify(afterLogin),
  shot: (await shoot('90-登录后企业总览')).file,
});

// C2. 上报列表 → 点击表格行 → 详情弹窗
await loadRoute('ENTERPRISE', '/ent/reports');
const drawer = await cdp.evaluate(`(function () {
  var row = document.querySelector('tbody tr.clickable') || document.querySelector('tbody tr');
  if (!row) return { ok: false, why: '无数据行' };
  row.click();
  return { ok: true, cols: row.children.length };
})()`);
await sleep(2000);
const drawerState = await cdp.evaluate(`(function(){
  var d = document.querySelector('.mask .modal');
  var t = d ? (d.innerText || '').replace(/\\s+/g,' ').trim() : '';
  var h = d && d.querySelector('.modal-head h3');
  return { open: !!d, title: h ? h.innerText : '', textLen: t.length, head: t.slice(0, 90) };
})()`);
interactions.push({
  name: '排放上报 → 行点击打开详情弹窗',
  pass: drawer.ok && drawerState.open && drawerState.textLen > 200,
  detail: JSON.stringify({ click: drawer, panel: drawerState }),
  shot: (await shoot('91-上报详情弹窗')).file,
});

// C3. 区块链浏览器 → 点进区块详情
//     注意：浏览器页除「区块列表」外还有「存证记录」等表格，必须限定到区块表，
//     否则会点到存证记录的「数字护照」按钮（历史缺陷：依赖「第一个 tbody 即区块表」）。
await loadRoute(null, '/explorer');
const blk = await cdp.evaluate(`(function () {
```

```
  var rows = document.querySelectorAll('.tbl-blocks tbody tr');
  if (!rows.length) return { ok: false, why: '无区块行' };
  var t = rows[0].querySelector('a,button,.link,.block-idx');
  if (t) { t.click(); return { ok: true, via: 'link', label: (t.innerText||'').trim() }; }
  rows[0].click(); return { ok: true, via: 'row' };
})()`);
await sleep(2200);
const blkState = await cdp.evaluate(`(function(){
  var app = document.getElementById('app');
  var txt = (app.innerText||'').replace(/\\s+/g,' ');
  return {
    hash: location.hash,
    textLen: (app.innerText||'').length,
    err: (document.querySelector('.alertbar.err')||{innerText:''}).innerText,
    merkle: /Merkle/i.test(txt),
    hashes: document.querySelectorAll('.hashbox,.mono').length,
  };
})()`);
interactions.push({
  name: '区块链浏览器 → 区块详情（含 Merkle 根）',
  pass: blk.ok && /^#\/explorer\/block\//.test(blkState.hash) && blkState.err === '' && blkState.textLen > 400,
  detail: JSON.stringify({ click: blk, after: blkState }),
  shot: (await shoot('92-区块详情')).file,
});

// C4. 监管端预警处置
await loadRoute('REGULATOR', '/reg/alerts');
const alertBtn = await cdp.evaluate(`(function () {
  var btns = Array.prototype.filter.call(document.querySelectorAll('tbody button'), function (b) {
    return (b.innerText || '').trim() === '处置';
  });
  if (!btns.length) return { ok: false, why: '无处置按钮' };
  btns[0].click(); return { ok: true, label: (btns[0].innerText||'').trim() };
})()`);
await sleep(1800);
const alertState = await cdp.evaluate(`(function(){
  var d = document.querySelector('.mask .modal');
  var h = d && d.querySelector('.modal-head h3');
  return { open: !!d, title: h ? h.innerText : '', textLen: d ? (d.innerText||'').length : 0, hash: location.hash };
})()`);
interactions.push({
  name: '监管端预警 → 处置弹窗',
  pass: alertBtn.ok && alertState.open && alertState.textLen > 120,
  detail: JSON.stringify({ click: alertBtn, panel: alertState }),
  shot: (await shoot('93-预警处置弹窗')).file,
});

// C5. 链上存证核验弹窗（Merkle 证明 + 多重验签）
await loadRoute('ENTERPRISE', '/ent/reports');
const proofBtn = await cdp.evaluate(`(function () {
```

```
  var btns = Array.prototype.filter.call(document.querySelectorAll('tbody button'), function (b) {
    return (b.innerText || '').trim() === '存证';
  });
  if (!btns.length) return { ok: false, why: '无存证按钮' };
  btns[0].click(); return { ok: true, label: (btns[0].innerText||'').trim() };
})()`);
await sleep(2600);
const proofState = await cdp.evaluate(`(function(){
  var d = document.querySelector('.mask .modal');
  if (!d) return { open: false };
  var txt = (d.innerText || '').replace(/\\s+/g,' ');
  return {
    open: true, textLen: txt.length,
    readouts: d.querySelectorAll('.readout,.ro').length,
    merkle: /Merkle/.test(txt),
    signed: /验签|签名/.test(txt),
    head: txt.slice(0, 140),
  };
})()`);
interactions.push({
  name: '企业端存证按钮 → 链上核验弹窗（验签 + Merkle 路径）',
  pass: proofBtn.ok && proofState.open && proofState.merkle && proofState.textLen > 200,
  detail: JSON.stringify({ click: proofBtn, panel: proofState }),
  shot: (await shoot('94-链上存证核验')).file,
});

/* ---- D. 汇总 ---- */
const passRoutes = results.filter((r) => r.pass).length;
const passInter = interactions.filter((r) => r.pass).length;
const summary = {
  generatedAt: new Date().toISOString(),
  base: BASE,
  chain: { height: chain.height, totalTxs: chain.totalTxs },
  routes: { total: results.length, pass: passRoutes, fail: results.length - passRoutes },
  interactions: { total: interactions.length, pass: passInter, fail: interactions.length - passInter },
  failures: results.filter((r) => !r.pass).map((r) => ({
    route: r.route, role: r.role, label: r.label,
    booting: r.probe.booting, textLen: r.probe.textLen,
    logs: r.probe.logs, errorBars: r.probe.errorBars, unresolved: r.probe.unresolved,
    warns: r.probe.warns,
    shot: r.shot.file,
  })),
  interactionFailures: interactions.filter((r) => !r.pass),
  details: results.map((r) => ({
    route: r.route, role: r.role, label: r.label, pass: r.pass,
    textLen: r.probe.textLen, tables: r.probe.tables, rows: r.probe.rows,
    cards: r.probe.cards, charts: r.probe.charts, hash: r.probe.hash,
    logs: r.probe.logs, errorBars: r.probe.errorBars, shot: r.shot.file,
  })),
  interactions,
```

```
  };
  fs.writeFileSync(path.join(OUT, 'ui-report.json'), JSON.stringify(summary, null, 2), 'utf8');

  const md = [
    '# UI 巡检报告（无头 Edge + CDP 实测）',
    '',
    `- 生成时间：${new Date().toLocaleString('zh-CN')}`,
    `- 站点：${BASE}  链高度：${chain.height}  累计存证：${chain.totalTxs}`,
    `- 路由渲染：**${passRoutes}/${results.length}** 通过`,
    `- 关键交互：**${passInter}/${interactions.length}** 通过`,
    '',
    '## 一、路由渲染明细',
    '',
    '| # | 角色 | 路由 | 页面 | 结果 | 文本量 | 表格 | 图表 | 控制台异常 |',
    '|---|------|------|------|------|--------|------|------|------------|',
    ...results.map((r, i) => `| ${i + 1} | ${r.role || '公众'} | \`${r.route}\` | ${r.label} | ${r.pass ? '✅' : '❌'} | ${
    '## 二、关键交互',
    '',
    '| 交互 | 结果 | 细节 | 截图 |',
    '|------|------|------|------|',
    ...interactions.map((x) => `| ${x.name} | ${x.pass ? '✅' : '❌'} | \`${x.detail}\` | \`${x.shot}\` |`),
    '',
  ];
  if (summary.failures.length) {
    md.push('## 三、失败明细', '');
    summary.failures.forEach((f) => {
      md.push(`### ${f.role || '公众'} ${f.route}（${f.label}）`, '');
      md.push(`- 仍在启动屏：${f.booting}  文本量：${f.textLen}  截图：\`${f.shot}\``);
      if (f.unresolved && f.unresolved.length) md.push('- 未解析标签：' + f.unresolved.map((s) => '`<' + s + '>`').join(' '))
      if (f.warns && f.warns.length) md.push('- 警告：\n' + f.warns.map((l) => `  - ${l.msg}`).join('\n'));
      if (f.errorBars && f.errorBars.length) md.push('- 页面错误条：' + f.errorBars.map((s) => '`' + s + '`').join('；'));
      if (f.logs && f.logs.length) md.push('- 控制台：\n' + f.logs.map((l) => `  - [${l.lvl}] ${l.msg}`).join('\n'));
      md.push('');
    });
  }
  if (summary.interactionFailures.length) {
    md.push('## 四、交互失败明细', '');
    summary.interactionFailures.forEach((f) => md.push(`- **${f.name}** → \`${f.detail}\``));
    md.push('');
  }
  fs.writeFileSync(path.join(OUT, 'ui-report.md'), md.join('\n'), 'utf8');

  console.log(`\n===== 路由 ${passRoutes}/${results.length}  交互 ${passInter}/${interactions.length} =====`);
  console.log('报告：artifacts/ui-report.md');
  summary.failures.forEach((f) => console.log(`FAIL ${f.role || 'PUBLIC'} ${f.route} logs=${JSON.stringify(f.logs)}`));

  try { ws.close(); } catch (e) { /* ignore */ }
  await killEdge(proc, profile);
  process.exit(0);
}
```

```
main().catch((e) => {
  console.error('巡检脚本异常：', e && e.stack || e);
  process.exit(1);
});

/* ================================================================
 * 文件：scripts/smoke-test.js
 * ================================================================ */
/**
 * smoke-test.js —— 接口冒烟测试 / 交付自检脚本
 * ----------------------------------------------------------------
 * 依次以四种角色登录，遍历全部核心接口，逐条校验 HTTP 状态与业务 code。
 * 用法： node scripts/smoke-test.js [baseUrl]
 */
'use strict';

const BASE = process.argv[2] || 'http://127.0.0.1:8300';

let pass = 0, fail = 0;
const failures = [];

async function api(method, path, { token, body } = {}) {
  const res = await fetch(BASE + path, {
    method,
    headers: {
      'Content-Type': 'application/json',
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
    },
    body: body ? JSON.stringify(body) : undefined,
  });
  let json = null;
  try { json = await res.json(); } catch (e) { /* ignore */ }
  return { status: res.status, json };
}

async function check(name, method, path, opts = {}, expectCode = 0) {
  try {
    const r = await api(method, path, opts);
    const bizOk = r.json && r.json.code === expectCode;
    if (r.status < 400 && bizOk) {
      pass++;
      const size = JSON.stringify(r.json.data || {}).length;
      console.log(`  ✓ ${name.padEnd(38)} ${String(r.status).padEnd(4)} ${size}B`);
    } else {
      fail++;
      failures.push(`${name} → HTTP ${r.status} / code ${r.json && r.json.code} / ${r.json && r.json.message}`);
      console.log(`  ✗ ${name.padEnd(38)} ${r.status} ${(r.json && r.json.message) || ''}`);
    }
    return r.json;
```

```
  } catch (e) {
    fail++;
    failures.push(`${name} → ${e.message}`);
    console.log(`  ✗ ${name.padEnd(38)} ${e.message}`);
    return null;
  }
}

async function login(username, password) {
  const r = await api('POST', '/api/auth/login', { body: { username, password } });
  if (r.json && r.json.code === 0) return r.json.data;
  throw new Error(`登录失败 ${username}: ${r.json && r.json.message}`);
}

(async () => {
  console.log(`\n═══ 碳链通 CarbonChain Hub 接口自检 · ${BASE} ═══\n`);

  /* ---- 公共 ---- */
  console.log('【公共接口】');
  await check('健康检查', 'GET', '/api/health');
  await check('演示账号列表', 'GET', '/api/auth/demo-accounts');
  await check('公示大屏', 'GET', '/api/public/dashboard');
  await check('公示企业名录', 'GET', '/api/public/enterprises?page=1&size=5');
  await check('公示公告', 'GET', '/api/public/notices?page=1&size=5');
  await check('碳价行情', 'GET', '/api/public/carbon-price?days=90');

  /* ---- 区块链浏览器 ---- */
  console.log('\n【区块链浏览器】');
  await check('链概况', 'GET', '/api/chain/stats');
  await check('区块列表', 'GET', '/api/chain/blocks?page=1&size=5');

  /* 区块索引不是从 0 连续编号的（创世块的 block_index 由链的构建方式决定），
   * 因此先取列表里真实存在的区块索引，再做详情/篡改测试，避免硬编码 10 在某些
   * 重建后的链上不存在。
   * 注意：列表接口返回的是库字段 block_index，详情接口返回的才是转换后的 index。 */
  const blkList = await check('区块列表(探测索引)', 'GET', '/api/chain/blocks?page=1&size=5');
  const blkRows = (blkList && blkList.data && blkList.data.rows) || [];
  const firstRow = blkRows[0] || {};
  const probeIndex = Number(firstRow.block_index !== undefined ? firstRow.block_index : firstRow.index);
  const tamperIndex = Number.isFinite(probeIndex) ? probeIndex : 1;

  const b = await check(`区块详情(#${tamperIndex})`, 'GET', `/api/chain/blocks/${tamperIndex}`);
  await check('交易列表', 'GET', '/api/chain/txs?page=1&size=5');
  await check('交易详情', 'GET', `/api/chain/tx/${b && b.data && b.data.transactions[0] ? b.data.transactions[0].tx_id : 'x'}
  await check('交易池', 'GET', '/api/chain/pool');
  await check('全链完整性校验', 'GET', '/api/chain/validate');
  await check('地址存证查询', 'GET', '/api/chain/address/' + (b && b.data && b.data.transactions[0] ? b.data.transactions[0].

  /* ---- 企业端 ---- */
  console.log('\n【企业端】');
```

```
const ent = await login('ent001', '123456');
const et = ent.token;
await check('企业概览', 'GET', '/api/enterprise/overview', { token: et });
const reports = await check('上报单列表', 'GET', '/api/enterprise/reports?page=1&size=20', { token: et });
const rows = (reports && reports.data && reports.data.rows) || [];
/* 挑一条【已上链】的上报单做存证详情测试：不是所有上报单都已完成上链，
 * 取第一条可能命中未上链的草稿，导致误报失败。 */
const chained = rows.find((r) => r.chain_tx_id || r.chainTxId || r.chain_block_index !== null && r.chain_block_index !== u
const rid = (chained || rows[0] || {}).id || 1;
await check('上报单详情', 'GET', `/api/enterprise/reports/${rid}`, { token: et });
await check('链上存证详情', 'GET', `/api/enterprise/reports/${rid}/chain`, { token: et });
await check('配额账户', 'GET', '/api/enterprise/quota', { token: et });
await check('我的委托', 'GET', '/api/enterprise/orders?page=1&size=5', { token: et });
await check('市场订单簿', 'GET', '/api/enterprise/orderbook', { token: et });
await check('我的成交', 'GET', '/api/enterprise/deals?page=1&size=5', { token: et });
await check('数据分析', 'GET', '/api/enterprise/analytics', { token: et });
await check('链上身份', 'GET', '/api/enterprise/wallet', { token: et });
await check('通知公告', 'GET', '/api/enterprise/notices?page=1&size=5', { token: et });

// 写操作：新建上报 → 提交 → 校验上链
// 幂等处理：同一企业同一「年度+季度」只允许一张上报单，
// 因此先取已存在的周期，再挑一个空位（优先放在下一年度，避免污染本年演示数据）。
const rnd = Math.floor(Math.random() * 900) + 100;
const listed = await api('GET', '/api/enterprise/reports?page=1&size=200', { token: et });
const myReports = (listed.json && listed.json.data && listed.json.data.rows) || [];
const taken = new Set(myReports.map((r) => r.year + '-' + r.quarter));
let slot = null;
for (const y of [Number(process.env.SMOKE_YEAR) || 2027, 2028, 2029, 2030]) {
  for (const q of [1, 2, 3, 4]) {
    if (!taken.has(y + '-' + q)) { slot = { year: y, quarter: q }; break; }
  }
  if (slot) break;
}
if (!slot) {
  console.log('⚠ 新建并提交上报（上链）  SKIP  该企业 2027–2030 年全部季度均已有上报单');
} else {
  const created = await check('新建并提交上报（上链）', 'POST', '/api/enterprise/reports', {
    token: et,
    body: {
      year: slot.year, quarter: slot.quarter, dataSource: '在线监测', submit: true,
      items: [{ energyType: '原煤', amount: 12000 + rnd }, { energyType: '电力', amount: 3200 + rnd }],
    },
  });
  if (created && created.data) {
    const nr = await check('新上报单链上存证核查', 'GET', `/api/enterprise/reports/${created.data.reportId}/chain`, { token
    if (nr && nr.data) {
      const v = nr.data.verify;
      console.log(`     ↳ 签名验签=${v.signatureValid} 数据完整=${v.dataIntact} 链下一致=${v.dbMatchesSnapshot} 区块#${nr.da
    }
  }
```

```
}
// 挂单撮合：故意报一个远离市场的买入价，使其无法立即成交，便于验证撤单
const order = await check('创建挂单（进入撮合队列）', 'POST', '/api/enterprise/orders', {
  token: et, body: { side: 'BUY', price: 1, amount: 300 },
});
if (order && order.data && order.data.orderId) {
  await check('撤销挂单', 'POST', `/api/enterprise/orders/${order.data.orderId}/cancel`, { token: et });
}

/* ---- 核查机构端 ---- */
console.log('\n【核查机构端】');
const ver = await login('ver001', '123456');
const vt = ver.token;
await check('机构概览', 'GET', '/api/verifier/overview', { token: vt });
const tasks = await check('核查任务列表', 'GET', '/api/verifier/tasks?page=1&size=5', { token: vt });
await check('核查任务详情', 'GET', `/api/verifier/tasks/${tasks && tasks.data && tasks.data.rows[0] ? tasks.data.rows[0].id
await check('已出具报告', 'GET', '/api/verifier/reports?page=1&size=5', { token: vt });
await check('机构链上身份', 'GET', '/api/verifier/wallet', { token: vt });
await check('通知公告', 'GET', '/api/verifier/notices?page=1&size=5', { token: vt });

/* ---- 监管端 ---- */
console.log('\n【监管端】');
const reg = await login('reg001', '123456');
const rt = reg.token;
await check('监管驾驶舱', 'GET', '/api/regulator/overview', { token: rt });
await check('企业名录', 'GET', '/api/regulator/enterprises?page=1&size=5', { token: rt });
await check('名录筛选项', 'GET', '/api/regulator/enterprises/filters', { token: rt });
await check('企业详情', 'GET', '/api/regulator/enterprises/1', { token: rt });
await check('配额分配流水', 'GET', '/api/regulator/allocations?page=1&size=5', { token: rt });
await check('交易监管', 'GET', '/api/regulator/trades?page=1&size=5', { token: rt });
await check('预警中心', 'GET', '/api/regulator/alerts?page=1&size=5', { token: rt });
await check('统计分析', 'GET', '/api/regulator/statistics', { token: rt });
await check('操作审计', 'GET', '/api/regulator/logs?page=1&size=5', { token: rt });
await check('系统概况', 'GET', '/api/regulator/system', { token: rt });
await check('公告管理', 'GET', '/api/regulator/notices?page=1&size=5', { token: rt });
await check('下发配额（上链）', 'POST', '/api/regulator/allocations', {
  token: rt, body: { entId: 2, year: 2026, amount: 1000, allocType: 'FREE', allocMethod: '行业基准线法', baseline: 1.25 },
});

/* ---- 篡改演示 ---- */
console.log('\n【区块链防篡改演示】');
const t1 = await check(`模拟篡改区块 #${tamperIndex}`, 'POST', '/api/chain/tamper', { token: rt, body: { blockIndex: tamper
if (t1 && t1.data) {
  console.log(`     ↳ 篡改后校验：valid=${t1.data.validation.valid} 异常数=${t1.data.validation.errors.length}`);
  console.log(`     ↳ 首条异常：${t1.data.validation.errors[0] ? t1.data.validation.errors[0].message : '-'}`);
}
const t2 = await check('修复链（按共识重新出块）', 'POST', '/api/chain/repair', { token: rt, body: { fromIndex: tamperIndex
if (t2 && t2.data) {
  console.log(`     ↳ 修复 ${t2.data.repaired.length} 个区块，重新出块（PoA 重新签名）总耗时 ${t2.data.repaired.reduce((a, b)
  console.log(`     ↳ 修复后校验：valid=${t2.data.validation.valid} 异常数=${t2.data.validation.errors.length}`);
```

```
    console.log(`      ↳ 注意：重算后整条后缀的区块哈希已全部改变 —— 篡改会使全链哈希失效并被各成员节点拒收`);
  }

  console.log('\n' + '='.repeat(64));
  console.log(`  测试用例 ${pass + fail} 条 · 通过 ${pass} · 失败 ${fail}`);
  if (failures.length) {
    console.log('  失败明细：');
    failures.forEach((f) => console.log('    - ' + f));
  } else {
    console.log('  ✅ 全部接口自检通过');
  }
  console.log('='.repeat(64) + '\n');
  process.exit(fail ? 1 : 0);
})().catch((e) => { console.error(e); process.exit(1); });

/* ================================================================
 * 文件：scripts/health-check.js
 * ================================================================ */
/* scripts/health-check.js —— 全项目数据真实性 + 全面性体检（零依赖） */
'use strict';
const fs = require('fs');
global.window = globalThis;  // 兜底：让浏览器专属数据文件在 Node 下可 require

const issues = [];
const ok = [];

/* 1. sources.js 因子台账 */
const S = require('../public/js/data/sources.js');
const pf = S.powerFactors;
if (pf) {
  ok.push('sources.powerFactors 存在，含 national/powerCo2/footprint/gridBaseline');
  if (pf.national && Math.abs(pf.national.value - 0.5703) < 0.0001) ok.push('全国电网平均因子 0.5703 ✓');
  else issues.push('FAIL 全国电网平均因子值不对：' + (pf.national && pf.national.value));
  if (pf.footprint && Math.abs(pf.footprint.value - 0.6205) < 0.0001) ok.push('电力碳足迹因子 0.6205 ✓');
  else issues.push('FAIL 电力碳足迹因子值不对：' + (pf.footprint && pf.footprint.value));
  if (pf.powerCo2 && Math.abs(pf.powerCo2.value - 0.5568) < 0.0001) ok.push('2021 年电力因子 0.5568 ✓');
  else issues.push('FAIL 2021 年电力因子值不对');
} else issues.push('FAIL sources.powerFactors 不存在');

if (S.fuelFactors) {
  let pass = 0, fail = 0;
  S.fuelFactors.forEach((f) => {
    // EF = NCV × CC × OF × 44/12，结果 tCO₂/单位；官方公布值 kgCO₂/单位 → ×1000
    const computed = f.ncv * f.cc * f.of * (44 / 12) * 1000;
    const dev = Math.abs(computed - f.ef) / f.ef * 100;
    if (f.traceLevel === 'C') return;  // 蒸汽为示例值，跳过
    if (dev < 0.5) pass++; else { fail++; issues.push('FAIL 燃料 ' + f.cn + ' 复算偏差 ' + dev.toFixed(3) + '%'); }
  });
  ok.push('IPCC 燃料因子复算：' + pass + ' 通过 / ' + fail + ' 失败（共 ' + (S.fuelFactors.length - 1) + ' 种，蒸汽示例值跳过）'
}
```

```
if (S.citations) ok.push('引用来源 ' + S.citations.length + ' 条');
else issues.push('FAIL sources.citations 不存在');

/* 2. industry.js */
const I = require('../public/js/data/industry.js');
if (I.buildDevices) {
  const devs = I.buildDevices();
  ok.push('industry 设备：' + devs.length + ' 台');
  if (devs.length < 12) issues.push('FAIL 设备数 < 12');
  let didOk = 0;
  devs.forEach((d) => { if (d.did && d.did.startsWith('did:')) didOk++; });
  ok.push('DID 格式合规：' + didOk + '/' + devs.length);
  const protos = new Set(devs.map((d) => d.protocol).filter(Boolean));
  ok.push('通信协议：' + Array.from(protos).join(' / '));
  const cemsDevs = devs.filter((d) => d.did && d.did.includes('cems'));
  ok.push('CEMS 设备：' + cemsDevs.length + ' 台');
}
if (I.buildBatches) {
  const batches = I.buildBatches();
  ok.push('industry 批次：' + batches.length + ' 个');
}

/* 3. 无 Math.random 检查（排除注释行，跟踪块注释状态） */
function codeHasMathRandom(src) {
  let inBlock = false;
  return src.split('\n').some((line) => {
    const trimmed = line.trim();
    if (inBlock) {
      if (trimmed.includes('*/')) inBlock = false;
      return false;
    }
    if (trimmed.startsWith('/*')) {
      if (!trimmed.includes('*/')) inBlock = true;
      return false;
    }
    if (trimmed.startsWith('//') || trimmed.startsWith('*')) return false;
    return /Math\.random\(\)/.test(line);
  });
}
const cemsSrc = fs.readFileSync('./public/js/data/cems.js', 'utf8');
if (codeHasMathRandom(cemsSrc)) issues.push('FAIL cems.js 代码行仍含 Math.random()');
else ok.push('cems.js 代码行已无 Math.random() ✓');
const rdeSrc = fs.readFileSync('./public/js/data/realDataEngine.js', 'utf8');
if (codeHasMathRandom(rdeSrc)) issues.push('FAIL realDataEngine.js 代码行含 Math.random()');
else ok.push('realDataEngine.js 代码行已无 Math.random() ✓');

const RDE = require('../public/js/data/realDataEngine.js');
if (RDE.ARCHIVE_CEMS_60 && RDE.ARCHIVE_CEMS_60.length === 60) ok.push('归档序列 60 帧 ✓');
else issues.push('FAIL 归档序列帧数不对');
```

```
/* 4. nucleus.js */
const N = require('../public/js/data/nucleus.js');
if (N.monteCarlo && N.enterpriseReconcile && N.leagueReconcile) ok.push('nucleus 核算函数齐全 ✓');
else issues.push('FAIL nucleus 核算函数缺失');

/* 5. SM3 官方向量 */
const SM3 = require('../public/js/data/sm3.js');
const v = SM3.sm3('abc');
if (v === '66c7f0f462eeedd9d1f2d46bdc10e4e24167c4875cf2f7a2297da02b8f4ba8e0') ok.push('SM3 官方向量 ✓');
else issues.push('FAIL SM3 abc 向量不对：' + v);

/* 6. carbonCalculator 三口径 */
const Calc = require('../public/js/data/carbonCalculator.js');
ok.push('因子三档：' + Calc.FACTOR_PRESETS.map((f) => f.key + '=' + f.getValue()).join(' / '));

/* 7. view + route 完整性 */
const views = ['login', 'screen', 'explorer', 'verify', 'cems', 'consortium', 'dynamic', 'credit', 'enterprise', 'verifier',
const vMissing = views.filter((vv) => !fs.existsSync('./public/js/views/' + vv + '.js'));
if (vMissing.length) issues.push('FAIL view 缺失：' + vMissing.join(','));
else ok.push('view ' + views.length + ' 个齐全 ✓');
const routes = ['auth', 'enterprise', 'verifier', 'regulator', 'chain', 'consortium', 'dynamic', 'credit', 'public', 'teleme
const rMissing = routes.filter((r) => !fs.existsSync('./server/routes/' + r + '.js'));
if (rMissing.length) issues.push('FAIL route 缺失：' + rMissing.join(','));
else ok.push('route ' + routes.length + ' 个齐全 ✓');

/* 8. index.html 脚本引用 */
const html = fs.readFileSync('./public/index.html', 'utf8');
const exp = ['vue.global.prod.js', 'echarts.min.js', 'core.js', 'components.js', 'sources.js', 'nucleus.js', 'sm3.js', 'real
const m = exp.filter((s) => !html.includes(s));
if (m.length) issues.push('FAIL index.html 缺脚本：' + m.join(','));
else ok.push('index.html 脚本引用完整 ✓');

/* 9. 数据文件 Node 兼容性（window 兜底） */
const dataFiles = ['sources.js', 'nucleus.js', 'sm3.js', 'realDataEngine.js', 'carbonCalculator.js', 'cems.js', 'industry.js
dataFiles.forEach((f) => {
  const src = fs.readFileSync('./public/js/data/' + f, 'utf8');
  if (/\}\)\(window\)/.test(src)) issues.push('WARN ' + f + ' 用 })(window) 无法在 Node 下 require（建议改 })(typeof window!=
});

/* 10. 服务端中间件完整性 */
const mws = ['auth.js', 'rate-limit.js', 'monitor.js', 'structured-log.js', 'canary.js'];
const mwMissing = mws.filter((mw) => !fs.existsSync('./server/middleware/' + mw));
if (mwMissing.length) issues.push('FAIL middleware 缺失：' + mwMissing.join(','));
else ok.push('middleware ' + mws.length + ' 个齐全 ✓');

/* 11. db schema 完整性 */
const schema = fs.existsSync('./db/01_schema.sql') ? fs.readFileSync('./db/01_schema.sql', 'utf8') : '';
if (schema) {
  const tables = (schema.match(/CREATE TABLE[^;]+/g) || []).length;
  ok.push('db schema 含 ' + tables + ' 张表');
```

```
  if (tables < 20) issues.push('FAIL schema 表数 < 20');
}

/* 12. package.json 依赖完整性 */
const pkg = JSON.parse(fs.readFileSync('./package.json', 'utf8'));
const deps = Object.keys(pkg.dependencies || {});
ok.push('dependencies: ' + deps.join(' / '));
if (!deps.includes('ws')) issues.push('FAIL 缺 ws 依赖');
if (!deps.includes('express')) issues.push('FAIL 缺 express 依赖');

/* 输出 */
console.log('=== 检测通过项 (' + ok.length + ') ===');
ok.forEach((m) => console.log('  OK  ' + m));
console.log('');
console.log('=== 待完善项 (' + issues.length + ') ===');
if (issues.length === 0) console.log('  无');
else issues.forEach((m) => console.log('  ' + m));
console.log('');
console.log('总计: ' + ok.length + ' 通过 / ' + issues.length + ' 待完善');
process.exit(issues.length > 0 ? 1 : 0);

/* ================================================================
 * 文件: db/01_schema.sql
 * ================================================================ */
-- ================================================================================
--  碳链通 CarbonChain Hub —— 企业碳足迹存证与碳配额交易平台
--  数据库结构定义脚本 (MySQL 8.0)
--  文件: db/01_schema.sql
--  说明: 本文件为纯 DDL，不含数据。数据由 server/scripts/init-db.js 生成，
--        并可导出为 db/02_seed_data.sql，方便直接导入还原。
-- ================================================================================

DROP DATABASE IF EXISTS carbon_chain;
CREATE DATABASE carbon_chain DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE carbon_chain;

SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;

-- ------------------------------------------------------------------------
-- 1. sys_user  统一用户账号表（四端共用一套账号体系，靠 role 区分身份）
-- ------------------------------------------------------------------------
CREATE TABLE sys_user (
  id            BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  username      VARCHAR(50)  NOT NULL                COMMENT '登录账号',
  password      VARCHAR(128) NOT NULL                COMMENT '口令摘要 = SHA256(salt + 明文)',
  salt          VARCHAR(32)  NOT NULL                COMMENT '口令盐值',
  real_name     VARCHAR(50)           DEFAULT NULL   COMMENT '姓名',
  role          VARCHAR(20)  NOT NULL                COMMENT '角色: ENTERPRISE 企业端 / VERIFIER 核查机构端 / REGULATOR 监管端
  org_id        BIGINT                DEFAULT NULL   COMMENT '所属组织ID(企业ID或核查机构ID)',
```

```
  org_name      VARCHAR(120)          DEFAULT NULL   COMMENT '所属组织名称快照',
  phone         VARCHAR(20)           DEFAULT NULL   COMMENT '手机号',
  email         VARCHAR(80)           DEFAULT NULL   COMMENT '邮箱',
  avatar        VARCHAR(255)          DEFAULT NULL   COMMENT '头像',
  status        TINYINT      NOT NULL DEFAULT 1      COMMENT '状态 1启用 0停用',
  last_login    DATETIME              DEFAULT NULL   COMMENT '最近登录时间',
  created_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_user_username (username),
  KEY idx_user_role (role),
  KEY idx_user_org (org_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='统一用户账号表';

-- ----------------------------------------------------------------------
-- 2. enterprise  控排企业名录
-- ----------------------------------------------------------------------
CREATE TABLE enterprise (
  id             BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  ent_code       VARCHAR(32)  NOT NULL                COMMENT '统一社会信用代码',
  ent_name       VARCHAR(120) NOT NULL                COMMENT '企业名称',
  short_name     VARCHAR(60)           DEFAULT NULL   COMMENT '企业简称',
  industry       VARCHAR(40)  NOT NULL                COMMENT '所属行业: 电力/钢铁/建材/化工/有色金属/造纸/石化',
  region         VARCHAR(40)  NOT NULL                COMMENT '所在省级行政区',
  city           VARCHAR(40)           DEFAULT NULL   COMMENT '所在城市',
  scale          VARCHAR(20)           DEFAULT NULL   COMMENT '规模: 大型/中型/小型',
  legal_person   VARCHAR(50)           DEFAULT NULL   COMMENT '法定代表人',
  contact_phone  VARCHAR(20)           DEFAULT NULL   COMMENT '联系电话',
  reg_capital    DECIMAL(18,2)         DEFAULT 0      COMMENT '注册资本(万元)',
  annual_output  DECIMAL(18,2)         DEFAULT 0      COMMENT '年产值(万元)',
  headcount      INT                   DEFAULT 0      COMMENT '员工人数',
  wallet_address VARCHAR(80)           DEFAULT NULL   COMMENT '链上钱包地址(secp256k1公钥压缩后派生)',
  pub_key        TEXT                  DEFAULT NULL   COMMENT '钱包公钥 PEM',
  credit_score   INT          NOT NULL DEFAULT 100    COMMENT '碳信用分(0-100)',
  status         VARCHAR(20)  NOT NULL DEFAULT 'ACTIVE' COMMENT '状态: ACTIVE 正常 / WATCH 关注 / SUSPENDED 暂停交易',
  created_at     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '纳入控排时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_ent_code (ent_code),
  KEY idx_ent_industry (industry),
  KEY idx_ent_region (region)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='控排企业名录表';

-- ----------------------------------------------------------------------
-- 3. verifier_org  第三方核查机构
-- ----------------------------------------------------------------------
CREATE TABLE verifier_org (
  id           BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  org_code     VARCHAR(32)  NOT NULL                COMMENT '机构备案编号',
  org_name     VARCHAR(120) NOT NULL                COMMENT '机构名称',
  region       VARCHAR(40)  NOT NULL                COMMENT '所在省份',
  license_no   VARCHAR(60)           DEFAULT NULL   COMMENT '资质证书编号',
```

```
  director     VARCHAR(50)           DEFAULT NULL   COMMENT '负责人',
  contact_phone VARCHAR(20)          DEFAULT NULL   COMMENT '联系电话',
  staff_count  INT          NOT NULL DEFAULT 0      COMMENT '专职核查员数量',
  level        VARCHAR(20)  NOT NULL DEFAULT 'B'    COMMENT '资质等级: A / B / C',
  done_tasks   INT          NOT NULL DEFAULT 0      COMMENT '累计完成核查任务数',
  pass_rate    DECIMAL(5,2) NOT NULL DEFAULT 0      COMMENT '报告一次性通过率(%)',
  rating       DECIMAL(3,1) NOT NULL DEFAULT 5.0    COMMENT '综合评级(0-5)',
  wallet_address VARCHAR(80)         DEFAULT NULL   COMMENT '链上钱包地址',
  pub_key      TEXT                  DEFAULT NULL   COMMENT '机构签名公钥 PEM（用于核验其出具的核查报告存证）',
  status       VARCHAR(20)  NOT NULL DEFAULT 'ACTIVE' COMMENT '状态',
  created_at   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '备案时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_verifier_code (org_code),
  KEY idx_verifier_region (region)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='第三方核查机构表';

-- ------------------------------------------------------------------------
-- 4. emission_report  企业碳排放数据上报单（Scope1 直接排放 + Scope2 间接排放）
-- ------------------------------------------------------------------------
CREATE TABLE emission_report (
  id               BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  report_no        VARCHAR(40)  NOT NULL                COMMENT '上报单号',
  ent_id           BIGINT       NOT NULL                COMMENT '企业ID',
  year             INT          NOT NULL                COMMENT '核算年度',
  quarter          INT          NOT NULL                COMMENT '核算季度 1-4',
  period           VARCHAR(10)  NOT NULL                COMMENT '报告期 形如 2025Q3',
  scope1_emission  DECIMAL(14,2) NOT NULL DEFAULT 0     COMMENT '范围一排放量(tCO2e)',
  scope2_emission  DECIMAL(14,2) NOT NULL DEFAULT 0     COMMENT '范围二排放量(tCO2e)',
  total_emission   DECIMAL(14,2) NOT NULL DEFAULT 0     COMMENT '排放总量(tCO2e)',
  energy_consumption DECIMAL(14,2) NOT NULL DEFAULT 0   COMMENT '综合能耗(吨标准煤)',
  output_value     DECIMAL(16,2) NOT NULL DEFAULT 0     COMMENT '报告期产值(万元)',
  intensity        DECIMAL(12,4) NOT NULL DEFAULT 0     COMMENT '排放强度(tCO2e/万元)',
  yoy_rate         DECIMAL(8,2)  NOT NULL DEFAULT 0     COMMENT '同比变化率(%)',
  data_source      VARCHAR(40)           DEFAULT NULL   COMMENT '数据来源: 在线监测/物料衡算/排放因子',
  status           VARCHAR(20)  NOT NULL DEFAULT 'DRAFT' COMMENT '状态: DRAFT 草稿 / SUBMITTED 已提交待核查 / VERIFYING 核查中
  verifier_id      BIGINT                DEFAULT NULL   COMMENT '承接受托的核查机构ID',
  submit_time      DATETIME              DEFAULT NULL   COMMENT '提交时间',
  verify_time      DATETIME              DEFAULT NULL   COMMENT '核查完成时间',
  chain_block_index INT                 DEFAULT NULL   COMMENT '存证所在区块高度',
  chain_tx_id      VARCHAR(80)           DEFAULT NULL   COMMENT '存证交易号',
  attachment       VARCHAR(255)          DEFAULT NULL   COMMENT '佐证材料',
  remark           VARCHAR(500)          DEFAULT NULL   COMMENT '备注',
  created_at       DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_report_no (report_no),
  KEY idx_report_ent (ent_id),
  KEY idx_report_status (status),
  KEY idx_report_period (period)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='企业碳排放数据上报单';
```

```
-- ----------------------------------------------------------------------
-- 5. emission_item  上报单的能源消耗明细行
-- ----------------------------------------------------------------------
CREATE TABLE emission_item (
  id           BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  report_id    BIGINT       NOT NULL                COMMENT '所属上报单ID',
  ent_id       BIGINT       NOT NULL                COMMENT '企业ID(冗余,便于统计)',
  energy_type  VARCHAR(30)  NOT NULL                COMMENT '能源品种: 原煤/焦炭/天然气/柴油/燃料油/电力/热力',
  scope        TINYINT      NOT NULL DEFAULT 1      COMMENT '排放范围 1直接 2间接',
  amount       DECIMAL(16,3) NOT NULL DEFAULT 0     COMMENT '消耗量',
  unit         VARCHAR(10)  NOT NULL DEFAULT 't'    COMMENT '计量单位',
  factor       DECIMAL(14,6) NOT NULL DEFAULT 0     COMMENT '排放因子',
  co2          DECIMAL(14,3) NOT NULL DEFAULT 0     COMMENT '折算排放量(tCO2e)',
  calc_method  VARCHAR(30)           DEFAULT NULL   COMMENT '计算方法',
  created_at   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  PRIMARY KEY (id),
  KEY idx_item_report (report_id),
  KEY idx_item_ent (ent_id),
  KEY idx_item_type (energy_type)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='碳排放上报明细行';

-- ----------------------------------------------------------------------
-- 6. verify_task  核查任务（企业与核查机构的委托关系）
-- ----------------------------------------------------------------------
CREATE TABLE verify_task (
  id            BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  task_no       VARCHAR(40)  NOT NULL                COMMENT '任务编号',
  report_id     BIGINT       NOT NULL                COMMENT '关联上报单ID',
  ent_id        BIGINT       NOT NULL                COMMENT '申请企业ID',
  verifier_id   BIGINT       NOT NULL                COMMENT '承接核查机构ID',
  verifier_user VARCHAR(50)           DEFAULT NULL   COMMENT '主责核查员账号',
  task_type     VARCHAR(20)  NOT NULL DEFAULT 'ROUTINE' COMMENT '任务类型: ROUTINE 常规核查 / RECHECK 复查 / RANDOM 抽查',
  priority      VARCHAR(10)  NOT NULL DEFAULT 'NORMAL'  COMMENT '优先级 HIGH/NORMAL/LOW',
  status        VARCHAR(20)  NOT NULL DEFAULT 'PENDING' COMMENT '状态: PENDING 待接单 / PROCESSING 核查中 / DONE 已完成 / REJE
  assign_time   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '派单时间',
  deadline      DATETIME              DEFAULT NULL   COMMENT '要求完成时间',
  finish_time   DATETIME              DEFAULT NULL   COMMENT '实际完成时间',
  reject_reason VARCHAR(500)          DEFAULT NULL   COMMENT '驳回原因',
  created_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_task_no (task_no),
  KEY idx_task_verifier (verifier_id),
  KEY idx_task_status (status),
  KEY idx_task_ent (ent_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='第三方核查任务表';

-- ----------------------------------------------------------------------
-- 7. verify_report  核查报告（核查结论上链存证）
-- ----------------------------------------------------------------------
CREATE TABLE verify_report (
```

```
  id              BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  report_no       VARCHAR(40)  NOT NULL                COMMENT '核查报告编号',
  task_id         BIGINT       NOT NULL                COMMENT '关联核查任务ID',
  emission_report_id BIGINT    NOT NULL                COMMENT '关联上报单ID',
  ent_id          BIGINT       NOT NULL                COMMENT '被核查企业ID',
  verifier_id     BIGINT       NOT NULL                COMMENT '出具机构ID',
  declared_value  DECIMAL(14,2) NOT NULL DEFAULT 0     COMMENT '企业申报值(tCO2e)',
  verified_value  DECIMAL(14,2) NOT NULL DEFAULT 0     COMMENT '核查认定值(tCO2e)',
  deviation_rate  DECIMAL(8,2) NOT NULL DEFAULT 0      COMMENT '偏差率(%)',
  conclusion      VARCHAR(20)  NOT NULL DEFAULT 'PASS' COMMENT '结论: PASS 通过 / FAIL 不通过 / CONDITIONAL 有条件通过',
  opinion         VARCHAR(500)          DEFAULT NULL   COMMENT '核查意见',
  auditor         VARCHAR(50)           DEFAULT NULL   COMMENT '主核查员',
  seal_hash       VARCHAR(80)           DEFAULT NULL   COMMENT '报告电子签章摘要',
  chain_block_index INT                 DEFAULT NULL   COMMENT '存证区块高度',
  chain_tx_id     VARCHAR(80)           DEFAULT NULL   COMMENT '存证交易号',
  issue_time      DATETIME              DEFAULT NULL   COMMENT '签发时间',
  created_at      DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_vreport_no (report_no),
  KEY idx_vreport_ent (ent_id),
  KEY idx_vreport_verifier (verifier_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='核查报告表';

-- ------------------------------------------------------------------------
-- 8. carbon_quota  企业碳配额账户（按年度一户一行）
-- ------------------------------------------------------------------------
CREATE TABLE carbon_quota (
  id              BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  ent_id          BIGINT       NOT NULL                COMMENT '企业ID',
  year            INT          NOT NULL                COMMENT '配额年度',
  total_allocated DECIMAL(14,2) NOT NULL DEFAULT 0     COMMENT '累计免费分配配额(tCO2e)',
  available       DECIMAL(14,2) NOT NULL DEFAULT 0     COMMENT '当前可用余额(tCO2e)',
  frozen          DECIMAL(14,2) NOT NULL DEFAULT 0     COMMENT '挂单冻结量(tCO2e)',
  used            DECIMAL(14,2) NOT NULL DEFAULT 0     COMMENT '已用于履约的配额(tCO2e)',
  bought          DECIMAL(14,2) NOT NULL DEFAULT 0     COMMENT '累计买入(tCO2e)',
  sold            DECIMAL(14,2) NOT NULL DEFAULT 0     COMMENT '累计卖出(tCO2e)',
  updated_at      DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_quota_ent_year (ent_id, year),
  KEY idx_quota_year (year)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='企业碳配额账户表';

-- ------------------------------------------------------------------------
-- 9. quota_allocation  配额分配流水（监管端下发，逐笔上链）
-- ------------------------------------------------------------------------
CREATE TABLE quota_allocation (
  id             BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  alloc_no       VARCHAR(40)  NOT NULL                COMMENT '分配单号',
  ent_id         BIGINT       NOT NULL                COMMENT '企业ID',
  year           INT          NOT NULL                COMMENT '年度',
```

```
  quota_amount  DECIMAL(14,2) NOT NULL DEFAULT 0     COMMENT '分配量(tCO2e)',
  alloc_type    VARCHAR(20)  NOT NULL DEFAULT 'FREE' COMMENT '类型: FREE 免费分配 / AUCTION 有偿竞价 / BUYBACK 政府回购',
  alloc_method  VARCHAR(30)           DEFAULT NULL   COMMENT '核定方法: 历史强度法 / 行业基准线法 / 历史总量法',
  baseline_intensity DECIMAL(12,4)     DEFAULT NULL   COMMENT '基准强度(tCO2e/万元)',
  operator      VARCHAR(50)           DEFAULT NULL   COMMENT '操作人',
  chain_block_index INT              DEFAULT NULL   COMMENT '上链区块高度',
  chain_tx_id   VARCHAR(80)           DEFAULT NULL   COMMENT '上链交易号',
  created_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '分配时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_alloc_no (alloc_no),
  KEY idx_alloc_ent (ent_id),
  KEY idx_alloc_year (year)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='碳配额分配流水表';

-- ------------------------------------------------------------------
-- 10. trade_order  碳配额交易挂单
-- ------------------------------------------------------------------
CREATE TABLE trade_order (
  id            BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  order_no      VARCHAR(40)  NOT NULL                COMMENT '委托单号',
  ent_id        BIGINT       NOT NULL                COMMENT '挂单企业ID',
  side          VARCHAR(10)  NOT NULL                COMMENT '方向: BUY 买入 / SELL 卖出',
  price         DECIMAL(10,2) NOT NULL DEFAULT 0     COMMENT '委托单价(元/吨)',
  amount        DECIMAL(14,2) NOT NULL DEFAULT 0     COMMENT '委托数量(tCO2e)',
  filled_amount DECIMAL(14,2) NOT NULL DEFAULT 0     COMMENT '已成交数量(tCO2e)',
  status        VARCHAR(20)  NOT NULL DEFAULT 'OPEN' COMMENT '状态: OPEN 挂单中 / PARTIAL 部分成交 / FILLED 已成交 / CANCELLED
  created_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '挂单时间',
  updated_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_order_no (order_no),
  KEY idx_order_ent (ent_id),
  KEY idx_order_status (status),
  KEY idx_order_side (side)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='碳配额交易挂单表';

-- ------------------------------------------------------------------
-- 11. trade_deal  成交记录
-- ------------------------------------------------------------------
CREATE TABLE trade_deal (
  id            BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  deal_no       VARCHAR(40)  NOT NULL                COMMENT '成交编号',
  buy_order_id  BIGINT                DEFAULT NULL   COMMENT '买方委托单ID',
  sell_order_id BIGINT                DEFAULT NULL   COMMENT '卖方委托单ID',
  buyer_ent_id  BIGINT       NOT NULL                COMMENT '买方企业ID',
  seller_ent_id BIGINT       NOT NULL                COMMENT '卖方企业ID',
  price         DECIMAL(10,2) NOT NULL DEFAULT 0     COMMENT '成交单价(元/吨)',
  amount        DECIMAL(14,2) NOT NULL DEFAULT 0     COMMENT '成交数量(tCO2e)',
  total_amount  DECIMAL(16,2) NOT NULL DEFAULT 0     COMMENT '成交金额(元)',
  fee           DECIMAL(12,2) NOT NULL DEFAULT 0     COMMENT '交易手续费(元)',
  deal_time     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '成交时间',
```

```
  chain_block_index INT              DEFAULT NULL   COMMENT '上链区块高度',
  chain_tx_id   VARCHAR(80)           DEFAULT NULL   COMMENT '上链交易号',
  status        VARCHAR(20)  NOT NULL DEFAULT 'SETTLED' COMMENT '状态: SETTLED 已清算 / REVOKED 已撤销',
  PRIMARY KEY (id),
  UNIQUE KEY uk_deal_no (deal_no),
  KEY idx_deal_buyer (buyer_ent_id),
  KEY idx_deal_seller (seller_ent_id),
  KEY idx_deal_time (deal_time)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='碳配额成交记录表';

-- ----------------------------------------------------------------------
-- 12. chain_block  区块链区块表（联盟链：哈希链 + Merkle 根 + PoA 授权出块）
-- ----------------------------------------------------------------------
CREATE TABLE chain_block (
  id          BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  block_index BIGINT       NOT NULL                COMMENT '区块高度',
  block_hash  VARCHAR(80)  NOT NULL                COMMENT '区块哈希(SHA-256)',
  prev_hash   VARCHAR(80)  NOT NULL                COMMENT '前一区块哈希',
  merkle_root VARCHAR(80)  NOT NULL                COMMENT '交易 Merkle 根',
  nonce       BIGINT       NOT NULL DEFAULT 0      COMMENT '随机数(历史 PoW 块为该块工作量证明的 nonce)',
  difficulty  INT          NOT NULL DEFAULT 0      COMMENT '难度(历史 PoW 块为前导零个数；PoA 块为 0)',
  consensus   VARCHAR(10)  NOT NULL DEFAULT 'POA'  COMMENT '共识类型: POA 授权证明 / POW 工作量证明(历史块)',
  proposer    VARCHAR(80)           DEFAULT NULL   COMMENT '本轮出块的联盟授权节点标识',
  proposer_sig VARCHAR(255)         DEFAULT NULL   COMMENT '出块节点对区块头的 ECDSA 签名',
  tx_count    INT          NOT NULL DEFAULT 0      COMMENT '打包交易数',
  miner       VARCHAR(80)           DEFAULT NULL   COMMENT '出块节点标识',
  size_bytes  INT          NOT NULL DEFAULT 0      COMMENT '区块大小(字节)',
  mine_time   BIGINT       NOT NULL DEFAULT 0      COMMENT '出块耗时(毫秒)',
  block_time  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '出块时间',
  created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '落库时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_block_index (block_index),
  KEY idx_block_hash (block_hash),
  KEY idx_block_time (block_time),
  KEY idx_block_proposer (proposer)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='联盟链区块表';

-- ----------------------------------------------------------------------
-- 13. chain_tx  链上交易（业务动作的可信存证）
-- ----------------------------------------------------------------------
CREATE TABLE chain_tx (
  id            BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  tx_id         VARCHAR(80)  NOT NULL                COMMENT '交易号(SHA-256 摘要)',
  tx_type       VARCHAR(30)  NOT NULL                COMMENT '类型: EMISSION_REPORT 排放上报 / VERIFY_REPORT 核查报告 / QUOTA_
  biz_no        VARCHAR(40)           DEFAULT NULL   COMMENT '关联业务单号',
  block_index   BIGINT                DEFAULT NULL   COMMENT '所属区块高度(NULL 表示在内存池中)',
  from_address  VARCHAR(80)           DEFAULT NULL   COMMENT '发起方钱包地址',
  to_address    VARCHAR(80)           DEFAULT NULL   COMMENT '接收方钱包地址',
  payload_hash  VARCHAR(80)  NOT NULL                COMMENT '业务数据 SHA-256 摘要',
  payload_json  TEXT                  DEFAULT NULL   COMMENT '业务数据快照',
```

```
  signature     VARCHAR(255)          DEFAULT NULL   COMMENT '发起方 ECDSA 数字签名(secp256k1)',
  pub_key       TEXT                  DEFAULT NULL   COMMENT '发起方公钥 PEM',
  nonce         BIGINT       NOT NULL DEFAULT 0      COMMENT '交易序号(防重放)',
  gas_fee       DECIMAL(12,4) NOT NULL DEFAULT 0      COMMENT '存证手续费(碳积分)',
  status        VARCHAR(20)  NOT NULL DEFAULT 'CONFIRMED' COMMENT '状态: PENDING 待打包 / CONFIRMED 已确认 / FAILED 失败',
  created_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '发起时间',
  confirmed_at  DATETIME              DEFAULT NULL   COMMENT '确认时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_tx_id (tx_id),
  KEY idx_tx_block (block_index),
  KEY idx_tx_type (tx_type),
  KEY idx_tx_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='链上存证交易表';

-- ------------------------------------------------------------------------
-- 13b. chain_payload_backup  链下明细篡改演示的原文备份
--      仅用于「篡改链下明细 → 第五道校验告警 → 一键还原」的演示闭环，
--      保证演示可反复进行、不污染链下业务数据。正式业务流程不写入本表。
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS chain_payload_backup (
  tx_id        VARCHAR(80) NOT NULL           COMMENT '被篡改的交易号',
  payload_json TEXT                 DEFAULT NULL COMMENT '篡改前的链下明文负载（原文备份）',
  backed_at    DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '备份时间',
  PRIMARY KEY (tx_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci COMMENT='链下明细篡改演示原文备份';

-- ------------------------------------------------------------------------
-- 14. alert_record  监管预警记录
-- ------------------------------------------------------------------------
CREATE TABLE alert_record (
  id          BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  alert_no    VARCHAR(40)  NOT NULL                COMMENT '预警编号',
  ent_id      BIGINT                DEFAULT NULL   COMMENT '关联企业ID',
  alert_type  VARCHAR(30)  NOT NULL                COMMENT '类型: OVER_QUOTA 超配额 / DATA_ABNORMAL 数据异常 / MISSING_REPORT
  level       VARCHAR(10)  NOT NULL DEFAULT 'MEDIUM' COMMENT '级别: LOW / MEDIUM / HIGH',
  title       VARCHAR(120) NOT NULL                COMMENT '预警标题',
  content     VARCHAR(500)          DEFAULT NULL   COMMENT '预警详情',
  metric      VARCHAR(60)           DEFAULT NULL   COMMENT '触发指标',
  status      VARCHAR(20)  NOT NULL DEFAULT 'OPEN' COMMENT '状态: OPEN 待处理 / HANDLING 处理中 / CLOSED 已关闭',
  handler     VARCHAR(50)           DEFAULT NULL   COMMENT '处理人',
  handle_note VARCHAR(500)          DEFAULT NULL   COMMENT '处理说明',
  created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '产生时间',
  closed_at   DATETIME              DEFAULT NULL   COMMENT '关闭时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_alert_no (alert_no),
  KEY idx_alert_ent (ent_id),
  KEY idx_alert_status (status),
  KEY idx_alert_level (level)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='监管预警记录表';
```

```
-- ----------------------------------------------------------------------
-- 15. audit_log  操作日志（等保与审计留痕）
-- ----------------------------------------------------------------------
CREATE TABLE audit_log (
  id         BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  user_id    BIGINT                DEFAULT NULL   COMMENT '操作人ID',
  username   VARCHAR(50)           DEFAULT NULL   COMMENT '操作人账号',
  role       VARCHAR(20)           DEFAULT NULL   COMMENT '操作人角色',
  module     VARCHAR(40)  NOT NULL                COMMENT '功能模块',
  action     VARCHAR(60)  NOT NULL                COMMENT '操作动作',
  target     VARCHAR(120)          DEFAULT NULL   COMMENT '操作对象',
  detail     VARCHAR(500)          DEFAULT NULL   COMMENT '操作详情',
  ip         VARCHAR(60)           DEFAULT NULL   COMMENT '来源IP',
  result     VARCHAR(10)  NOT NULL DEFAULT 'SUCCESS' COMMENT '结果 SUCCESS/FAIL',
  created_at DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '操作时间',
  PRIMARY KEY (id),
  KEY idx_log_user (user_id),
  KEY idx_log_module (module),
  KEY idx_log_time (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='系统操作日志表';

-- ----------------------------------------------------------------------
-- 16. notice  平台通知公告
-- ----------------------------------------------------------------------
CREATE TABLE notice (
  id           BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  title        VARCHAR(150) NOT NULL                COMMENT '标题',
  content      TEXT                  DEFAULT NULL   COMMENT '正文',
  notice_type  VARCHAR(20)  NOT NULL DEFAULT 'POLICY' COMMENT '类型: POLICY 政策法规 / MARKET 市场公告 / SYSTEM 系统通知',
  target_role  VARCHAR(20)  NOT NULL DEFAULT 'ALL'  COMMENT '目标角色 ALL/ENTERPRISE/VERIFIER/REGULATOR',
  publisher    VARCHAR(50)           DEFAULT NULL   COMMENT '发布人',
  is_top       TINYINT      NOT NULL DEFAULT 0      COMMENT '是否置顶',
  views        INT          NOT NULL DEFAULT 0      COMMENT '浏览量',
  publish_time DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '发布时间',
  PRIMARY KEY (id),
  KEY idx_notice_type (notice_type),
  KEY idx_notice_time (publish_time)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='平台通知公告表';

-- ----------------------------------------------------------------------
-- 17. chain_node  联盟链成员节点（准入白名单 + 出块权 + 副本状态）
--     联盟链的本质：参与方是"许可制"的成员，各成员节点共同出块并各自持链。
-- ----------------------------------------------------------------------
CREATE TABLE chain_node (
  id             BIGINT       NOT NULL AUTO_INCREMENT COMMENT '主键',
  node_id        VARCHAR(40)  NOT NULL                COMMENT '节点标识 如 node-regulator-01',
  node_name      VARCHAR(80)  NOT NULL                COMMENT '节点名称',
  org_type       VARCHAR(20)  NOT NULL                COMMENT '机构类型: REGULATOR 监管 / VERIFIER 核查 / ENTERPRISE 企业 / A
  org_name       VARCHAR(120)          DEFAULT NULL   COMMENT '所属机构名称',
  wallet_address VARCHAR(80)           DEFAULT NULL   COMMENT '节点钱包地址(链上身份)',
```

```
  public_key      TEXT                  DEFAULT NULL   COMMENT '节点公钥 PEM(用于验证该节点的出块签名)',
  status          VARCHAR(20)  NOT NULL DEFAULT 'PENDING' COMMENT '状态: PENDING 待审批 / ACTIVE 已授权 / REVOKED 已吊销',
  is_authorized   TINYINT      NOT NULL DEFAULT 0      COMMENT '是否具备出块权(联盟授权)',
  vote_weight     INT          NOT NULL DEFAULT 1      COMMENT '轮值权重(权重越高被选中出块的概率越大)',
  last_propose_at DATETIME              DEFAULT NULL   COMMENT '最近一次出块时间',
  proposed_blocks INT          NOT NULL DEFAULT 0      COMMENT '累计出块数',
  replica_height  BIGINT       NOT NULL DEFAULT -1     COMMENT '本地链副本已同步高度(-1 表示尚未同步)',
  replica_tip     VARCHAR(80)           DEFAULT NULL   COMMENT '本地副本链顶哈希',
  applied_at      DATETIME              DEFAULT NULL   COMMENT '申请加入时间',
  approved_at     DATETIME              DEFAULT NULL   COMMENT '审批通过时间',
  remark          VARCHAR(255)          DEFAULT NULL   COMMENT '备注/审批意见',
  created_at      DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_node_id (node_id),
  KEY idx_node_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='联盟链成员节点表';

-- ----------------------------------------------------------------------
-- 18. chain_node_replica  各成员节点的本地链副本
--     每同步一个区块插一行，用于证明"每个节点各自持有全量链副本"，
--     并可对比各节点链顶哈希是否与主链一致（可发现被篡改的节点）。
-- ----------------------------------------------------------------------
CREATE TABLE chain_node_replica (
  id          BIGINT      NOT NULL AUTO_INCREMENT COMMENT '主键',
  node_id     VARCHAR(40) NOT NULL                COMMENT '节点标识',
  block_index BIGINT      NOT NULL                COMMENT '已同步到的区块高度',
  block_hash  VARCHAR(80) NOT NULL                COMMENT '该高度区块哈希(副本中保存的值)',
  synced_at   DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '同步时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_replica_node_block (node_id, block_index),
  KEY idx_replica_node (node_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='联盟节点链副本表';

-- ----------------------------------------------------------------------
-- 19. cems_reading  在线监测(CEMS)实测读数
--     工业场景的"动态核算"来源：排口在线监测设备按小时推送烟气流量与
--     折算排放速率，替代人工季度填报，使核算从"周期性离线"变为"准实时滚动"。
-- ----------------------------------------------------------------------
CREATE TABLE cems_reading (
  id           BIGINT        NOT NULL AUTO_INCREMENT COMMENT '主键',
  ent_id       BIGINT        NOT NULL                COMMENT '企业ID',
  monitor_no   VARCHAR(40)   NOT NULL                COMMENT '监测点(排口)编号',
  energy_type  VARCHAR(30)   NOT NULL                COMMENT '能源品种/排放源',
  reading_time DATETIME      NOT NULL                COMMENT '采样时间',
  flow_value   DECIMAL(16,3) NOT NULL DEFAULT 0      COMMENT '瞬时排放速率(tCO2e/h)',
  cum_value    DECIMAL(18,3) NOT NULL DEFAULT 0      COMMENT '当期累计排放(tCO2e)',
  o2_content   DECIMAL(6,2)           DEFAULT NULL   COMMENT '烟气含氧量(%)',
  temperature  DECIMAL(6,2)           DEFAULT NULL   COMMENT '烟气温度(℃)',
  source       VARCHAR(20)   NOT NULL DEFAULT 'CEMS' COMMENT '数据来源 CEMS 在线监测 / MANUAL 人工补录',
  status       VARCHAR(20)   NOT NULL DEFAULT 'NORMAL' COMMENT '数据状态 NORMAL 正常 / ABNORMAL 异常',
```

```
  batch_no     VARCHAR(40)            DEFAULT NULL   COMMENT '采数批次号(同批次整体上链)',
  chain_tx_id  VARCHAR(80)            DEFAULT NULL   COMMENT '该批次上链存证交易号',
  created_at   DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '落库时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_cems_point (ent_id, monitor_no, reading_time),
  KEY idx_cems_ent_time (ent_id, reading_time),
  KEY idx_cems_time (reading_time),
  KEY idx_cems_batch (batch_no)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='CEMS 在线监测读数表（同一排口同一时刻唯一，采数幂等）';

-- ----------------------------------------------------------------------
-- 20. carbon_dynamic_account  企业动态核算账户（滚动核算结果）
--     由 CEMS 实测读数滚动计算：周期累计排放、期末预测、配额缺口、履约风险。
-- ----------------------------------------------------------------------
CREATE TABLE carbon_dynamic_account (
  id                  BIGINT        NOT NULL AUTO_INCREMENT COMMENT '主键',
  ent_id              BIGINT        NOT NULL                COMMENT '企业ID',
  year                INT           NOT NULL                COMMENT '核算年度',
  period              VARCHAR(10)   NOT NULL                COMMENT '当前核算周期 如 2026Q4',
  cumulative_emission DECIMAL(18,3) NOT NULL DEFAULT 0      COMMENT '本年度累计实测排放(tCO2e)',
  quota_allocated     DECIMAL(16,3) NOT NULL DEFAULT 0      COMMENT '本年度已分配配额(tCO2e)',
  quota_available     DECIMAL(16,3) NOT NULL DEFAULT 0      COMMENT '当前可用配额余额(tCO2e)',
  gap                 DECIMAL(18,3) NOT NULL DEFAULT 0      COMMENT '预计期末配额缺口(预测期末排放-已分配配额，正=预计超排，负=预
  predicted_eoy       DECIMAL(18,3) NOT NULL DEFAULT 0      COMMENT '按当前滚动排放速率预测的期末排放(tCO2e,抵销前)',
  offset_applied      DECIMAL(18,3) NOT NULL DEFAULT 0      COMMENT '已用碳信用抵销量(tCO2e)',
  emission_rate       DECIMAL(16,4) NOT NULL DEFAULT 0      COMMENT '滚动日均排放速率(tCO2e/天)',
  risk_level          VARCHAR(10)   NOT NULL DEFAULT 'LOW'  COMMENT '履约风险 LOW 低 / MEDIUM 中 / HIGH 高',
  data_points         INT           NOT NULL DEFAULT 0      COMMENT '参与核算的实测点数',
  last_reading_at     DATETIME                DEFAULT NULL  COMMENT '最近一次实测数据时间',
  updated_at          DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '核算更新时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_dyn_ent_year (ent_id, year),
  KEY idx_dyn_risk (risk_level)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='企业动态核算账户表';

-- ----------------------------------------------------------------------
-- 21. credit_asset  碳信用资产（国家核证自愿减排量 CCER 等）
--     除"碳配额"外的第二类碳资产：企业持有的自愿减排量，可用于抵销履约缺口。
-- ----------------------------------------------------------------------
CREATE TABLE credit_asset (
  id           BIGINT        NOT NULL AUTO_INCREMENT COMMENT '主键',
  ent_id       BIGINT        NOT NULL                COMMENT '持有企业ID',
  asset_no     VARCHAR(40)   NOT NULL                COMMENT '资产编号',
  asset_type   VARCHAR(30)   NOT NULL                COMMENT '资产类型 CCER 国家核证自愿减排量 / 林业碳汇 / 绿证',
  project_name VARCHAR(120)           DEFAULT NULL   COMMENT '减排项目名称',
  amount       DECIMAL(16,3) NOT NULL DEFAULT 0      COMMENT '持有量(tCO2e)',
  used_amount  DECIMAL(16,3) NOT NULL DEFAULT 0      COMMENT '已抵销使用量(tCO2e)',
  issue_year   INT                    DEFAULT NULL   COMMENT '签发年度',
  valid_until  DATE                   DEFAULT NULL   COMMENT '有效期截止',
  registry     VARCHAR(120)           DEFAULT NULL   COMMENT '登记机构',
```

```
  status       VARCHAR(20)   NOT NULL DEFAULT 'ACTIVE' COMMENT '状态 ACTIVE 可用 / USED 已用尽 / EXPIRED 过期',
  chain_block_index INT              DEFAULT NULL    COMMENT '签发存证区块高度',
  chain_tx_id  VARCHAR(80)            DEFAULT NULL   COMMENT '签发存证交易号',
  created_at   DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '落库时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_credit_asset_no (asset_no),
  KEY idx_credit_ent (ent_id, status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='碳信用资产表（配额之外的碳资产类型）';

-- ----------------------------------------------------------------------
-- 22. credit_offset  碳信用抵销使用流水
--     企业用碳信用抵销本年度配额缺口（受抵销比例上限约束），抵销即上链存证。
-- ----------------------------------------------------------------------
CREATE TABLE credit_offset (
  id           BIGINT        NOT NULL AUTO_INCREMENT COMMENT '主键',
  offset_no    VARCHAR(40)   NOT NULL                COMMENT '抵销单号',
  ent_id       BIGINT        NOT NULL                COMMENT '企业ID',
  year         INT           NOT NULL                COMMENT '抵销年度',
  asset_id     BIGINT        NOT NULL                COMMENT '使用的碳信用资产ID',
  amount       DECIMAL(16,3) NOT NULL                COMMENT '本次抵销量(tCO2e)',
  ratio_limit  DECIMAL(8,4)  NOT NULL DEFAULT 0.05   COMMENT '抵销比例上限(占配额)',
  quota_before DECIMAL(18,3) NOT NULL DEFAULT 0      COMMENT '抵销前预计缺口(tCO2e)',
  quota_after  DECIMAL(18,3) NOT NULL DEFAULT 0      COMMENT '抵销后预计缺口(tCO2e)',
  operator     VARCHAR(40)            DEFAULT NULL   COMMENT '经办人',
  chain_block_index INT              DEFAULT NULL    COMMENT '存证区块高度',
  chain_tx_id  VARCHAR(80)            DEFAULT NULL   COMMENT '存证交易号',
  status       VARCHAR(20)   NOT NULL DEFAULT 'CONFIRMED' COMMENT '状态',
  created_at   DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '落库时间',
  PRIMARY KEY (id),
  UNIQUE KEY uk_offset_no (offset_no),
  KEY idx_offset_ent (ent_id, year)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='碳信用抵销流水表';

SET FOREIGN_KEY_CHECKS = 1;
```